# CoNR-Miner: Self-Adaptive Co-Occurrence Nonoverlapping Sequential Rule Mining

Yan Li, Mengyao He, Jianguo Wei, *Senior Member, IEEE*, and Youxi Wu, *Senior Member, IEEE*

***Abstract*—Traditional sequential pattern mining (SPM) can discover all frequent patterns and sequential rule mining can discover all strong rules with high confidence based on the results of SPM. However, discovering all strong rules is meaningless, since users sometimes are interested in rules with same antecedent, i.e., co-occurrence rule. Previous co-occurrence nonoverlapping sequential rule mining only can discover strong co-occurrence rules with gap constraints on simple sequences composed of items. This paper explores self-adaptive co-occurrence nonoverlapping sequential rule (CoNR) mining with self-adaptive gaps on general sequences composed of itemsets and proposes a CoNR-Miner algorithm. In data preprocessing, CoNR-Miner constructs an integer mask dictionary to represent the original sequential database. In support calculation, we employ bitwise operations on the masks of subpatterns and items, significantly improving the computational efficiency. To reduce the number of candidate patterns, we propose a frequent binomial extension dictionary strategy to reduce candidate patterns. More importantly, we propose four pruning strategies to further prune candidate patterns. To evaluate the performance of CoNR-Miner, 16 competitive algorithms are conducted on eight databases. Experimental results on real databases demonstrate that CoNR-Miner is significantly faster than state-of-the-art algorithms. Furthermore, it exhibits excellent recommendation performance.**

***Index Terms*—Sequential pattern mining, sequential rule mining, self-adaptive gap, co-occurrence pattern, repetitive sequential pattern.**

## I. Introduction

SEQUATIAL pattern mining (SPM) is a significant data mining technique aimed at identifying frequent and potentially meaningful patterns within sequential databases [1]. To address diverse mining requirements, numerous SPM approaches have been proposed, including three-way SPM [2], [3], weak-gap strong SPM [4], periodic SPM [5], contrast SPM [6], [7], negative SPM [8], [9], target SPM [10], [11], order-preserving SPM [12], [13], and co-occurrence SPM [14] (similar to co-location pattern mining [15], [16]). Traditional SPM algorithms discover frequently occurring subsequences from itemset sequential databases and can be applied to mining customer purchase patterns. Suppose a customer first purchases item "$a$", then buys "$a$", "$b$", "$c$", and "$d$", followed by "$a$", then "$a$", "$b$", "$c$", and "$d$", then followed by "$a$", "$c$", and "$d$" twice, then "$d$" and "$e$", and finally "$d$". This customer's shopping sequence is, therefore, $\mathbf{s}_1 = (a)(abcd)(a)(abcd)(acd)(acd)(de)(d)$.

Classic SPM methods only consider whether a pattern occurs in a sequence, neglecting the number of times it actually occurs within the sequence [17], [18]. For example, in sequence $\mathbf{s}_1 = (a)(abcd)(a)(abcd)(acd)(acd)(de)(d)$, pattern $(ab)$ occurs twice, while traditional SPM only records it as occurring once. This disregard for pattern repetition within sequences risks overlooking potentially valuable information. To address this issue, gap constraint SPM has been proposed [19]. However, when a user's prior knowledge is insufficient, it is challenging to set appropriate gap constraints. In turn, inappropriate gap constraints hinder the effective extraction of valuable patterns. Consequently, adaptive gap constraints have been proposed to resolve this problem. Gap constraint SPM is categorized into four types: no condition [20], one-off condition [21], [22], nonoverlapping condition [19], [23], and disjoint condition [24]. Previous research indicated that nonoverlapping SPM avoids generating redundant patterns.

Nonoverlapping SPM methods [25] can identify all frequent patterns; however, it cannot directly serve prediction or recommendation tasks. Rule mining [26] can effectively uncover potential relationships between patterns by setting a minimum confidence threshold, i.e., $conf(\mathbf{p} \rightarrow \mathbf{q}) \geq mincf$, which means that if pattern $\mathbf{p}$ occurs in the sequence, pattern $\mathbf{q}$ is likely to appear afterward with a probability higher than or equal to a given confidence $mincf$. However, discovering all strong rules within databases holds limited practical significance. In real-world applications, such as recommendation systems, users often possess partial historical behavior (prefix patterns) and users only want to find the rules with the same antecedent $\mathbf{p}$, i.e., co-occurrence rule. Mining co-occurrence rules can not only filter irrelevant rules but also offer greater scenario adaptability. For example, by analyzing a user's recent behavior, one can precisely recommend his next likely purchasing items [27], significantly enhancing recommendation performance.

However, most existing co-occurrence rule mining methods are limited to single-item sequences, rely on manually specified gap constraints, and disregard pattern repetition within the sequences. In contrast, our method extends co-occurrence rule mining to general itemset sequences, employs adaptive gap

Received 23 December 2025; revised 1 May 2026; accepted 13 June 2026. Date of publication 19 June 2026; date of current version 8 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62372154. Recommended for acceptance by G. Cong. *(Corresponding author: Youxi Wu.)*

Yan Li and Mengyao He are with the School of Economics and Management, Hebei University of Technology, Tianjin 300400, China (e-mail: lywuc@163.com; hemengyao25@163.com).

Jianguo Wei is with the College of Intelligence and Computing, Tianjin University, Tianjin 300135, China (e-mail: jianguo@tju.edu.cn).

Youxi Wu is with the School of Artificial Intelligence, Hebei University of Technology, Tianjin 300400, China (e-mail: wuc567@163.com).

Digital Object Identifier 10.1109/TKDE.2026.3705564

TABLE I
SEQUENTIAL DATABASE $D$

| Sid | Sequence |
|---|---|
| $s_1$ | $(a)(abcd)(a)(abcd)(acd)(acd)(de)(d)$ |
| $s_2$ | $(af)(cde)(abcde)(abcdf)(abcdf)(c)(e)(cd)$ |
| $s_3$ | $(cdef)(d)(a)(bcdef)(ab)(g)$ |

constraints to eliminate the need for manual specification, and constructs co-occurrence rules suited for practical application scenarios. An illustrative example is as follows.

*Example 1:* Suppose we have a sequential database $D$, as shown in Table I, with a minimum support threshold of $minsup$=8. According to nonoverlapping SPM, there are 25 frequent patterns: $\{(a), (c), (d), (cd), (a)(a), (a)(c), (a)(d), (c)(a), (c)(c), (c)(d), (d)(a), (d)(c), (d)(d), (a)(cd), (c)(cd), (d)(cd), (cd)(a), (cd)(c), (cd)(d), (a)(a)(d), (a)(d)(d), (c)(d)(d), (d)(d)(d), (cd)(cd)$, and $(cd)(d)(d)\}$. When the minimum confidence threshold $mincf = 0.75$, there are 15 rules: $\{(c) \rightarrow (*d), (a) \rightarrow (a), (a) \rightarrow (c), (a) \rightarrow (d), (c) \rightarrow (c), (c) \rightarrow (d), (d) \rightarrow (d), (a)(c) \rightarrow (*d), (c)(c) \rightarrow (*d), (d)(c) \rightarrow (*d), (cd) \rightarrow (d), (a)(a) \rightarrow (d), (c)(d) \rightarrow (d), (cd)(c) \rightarrow (*d)$, and $(cd)(d) \rightarrow (d)\}$. Clearly, it is difficult for users to apply such a large number of frequent patterns and rules. However, if a prefix pattern $\mathbf{p}$=$(cd)$ is given, its frequent co-occurrence patterns and co-occurrence rules are significantly reduced. For example, $\mathbf{p}$=$(cd)$ appears 11 times in $D$, yielding a support of 11. Similarly, the support of $(cd)(d)$ in $D$ is 9, which is greater than $minsup$=8, making it a frequent co-occurrence pattern. $conf((cd) \rightarrow (d))$=9/11=0.82, which is greater than $mincf$=0.75, thus $(cd) \rightarrow (d)$ is a strong co-occurrence rule. Similarly, $(cd)(a), (cd)(c), (cd)(cd)$, and $(cd)(d)(d)$ are all co-occurrence patterns, and no other co-occurrence rule exists. Hence, there are only five frequent co-occurrence patterns with the prefix pattern $\mathbf{p}$=$(cd)$ and one co-occurrence rule, resulting in a significant reduction in the number of frequent patterns and rules discovered.

The main contributions of this paper are as follows:

1) To mine rules with same antecedent and consider pattern repetition in sequences with itemsets, we explore self-adaptive co-occurrence nonoverlapping sequential rule (CoNR) mining and propose a CoNR-Miner algorithm.
2) In support calculation, we propose an integer mask dictionary to store frequent pattern positions and calculate superpattern support using subpattern integer masks.
3) In candidate pattern generation, a frequent binomial extension dictionary strategy and four pruning strategies are proposed to reduce the number of candidate patterns.
4) Experiments confirm that CoNR-Miner outperforms all 16 competitive algorithms on eight databases and demonstrates superior recommendation capabilities.

The structure of this paper is as follows. Section II provides an overview of related work. Section III defines the problem. Section IV introduces the CoNR-Miner algorithm. Section V validates the performance of CoNR-Miner and Section VI concludes the paper.

## II. RELATED WORK

SPM constitutes a significant branch of data mining, aiming to extract subsequences (patterns) which satisfy specific conditions in sequential databases [18]. SPM has been extensively applied in behavior analysis [28], [29], feature extraction [7], [30], and bioinformatics research [3]. To address diverse requirements, various forms of SPM have been proposed. To consider utility effects [31], high utility SPM [32], [33], high utility negative SPM [34], [35] and high average utility SPM [36], [37] have been investigated. To reduce the number of mined patterns, top-$k$ SPM [38], [39], closed SPM [40], and maximal SPM [41] have been proposed. Moreover, targeted SPM [42] discovers superpatterns containing specific subpatterns, while co-occurrence SPM discovers superpatterns with same prefix subpatterns. These two methods can also reduce the number of mined patterns. Concurrently, SPM has been conducted for different database types, such as incremental databases [43], [44] and dynamic databases [45].

Traditional SPM focuses on whether a pattern occurs in a sequence, disregarding the repetition of pattern occurrence. Consequently, SPM with gap constraints have been proposed. However, setting appropriate gap constraints is challenging when users lack prior knowledge, which leads to the introduction of adaptive gap constraints. This approach is not target and fails to meet practical requirements. To address this issue, co-occurrence SPM algorithms have been proposed. These algorithms enhance the mining efficiency by only mining frequent patterns related to the given prefixes. For example, Kieu et al. [46] proposed a method for mining top-$k$ items that co-occur with specific patterns. Wang et al. [47] investigated the problem of mining top-$k$ closed co-occurrence patterns.

The aforementioned methods primarily focused on pattern discovery, and yet patterns alone cannot reveal relationships between them. Rule mining can effectively uncover relationships between patterns. The rule mining methods include various types, such as association rules [48], episode rules [49], sequential rules [26], and order-preserving rules [50]. To prevent redundant rule generation, streamlining strategies, such as maximum consequent and minimum antecedent [51], top-$k$ rules [48], and maximal rules [14], have been proposed. Furthermore, co-occurrence sequential pattern mining focuses on patterns with the same prefixes, to reduce the number of mined patterns [52]. Co-occurrence rule mining is based on co-occurrence SPM, which can further reveal relationships between prefix patterns and superpatterns, and MCoR-Miner was proposed to mine co-occurrence rules [14]. However, MCoR-Miner only can deal with single sequences with items and cannot deal with general complex sequences with itemsets, and the gaps are manually set and cannot be adaptively adjusted. To discover patterns from general complex sequences that are relevant to users' recent historical behavior, inspired by co-occurrence SPM [14] and repetitive nonoverlapping SPM [30], we propose self-adaptive co-occurrence nonoverlapping sequential rule (CoNR) mining to discover co-occurrence sequential rules in sequential databases.

Compared with existing co-occurrence rule mining methods, CoNR-Miner exhibits four conceptual distinctions: 1) it focuses

on itemset sequences rather than single-item sequences; 2) it employs adaptive nonoverlapping gap constraints without the need for manual specification; 3) it considers the repetition of pattern occurrences rather than merely judging based on whether pattern occurs; 4) it mines complete co-occurrence rules that directly associate prefix behaviors with subsequent items for recommendation tasks.

## III. Problem Definition

*Definition 1:* (Sequential database) Sequential database $D$ is a set of $k$ sequences, denoted by $\boldsymbol{D} = \{\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_k\}$. Each sequence in $D$ is composed of multiple itemsets, denoted by $\mathbf{s}_i = s_{i1}s_{i2}\cdots s_{ir}$, where $s_{ij}(1 \le j \le r)$ is an ordered itemset. The size of sequence $\mathbf{s}_i$ is the number of its itemsets, denoted by *size*($\mathbf{s}_i$)=$r$. The length of sequence $\mathbf{s}_i$ is the sum of the length of each itemset, i.e., $len(\mathbf{s}_i) = \sum_{j=1}^{r} len(s_{ij})$.

*Example 2:* We take $\mathbf{s}_1 = (a)(abcd)(a)(abcd)(acd)(acd)(de)(d)$ in Table I as an example, which has eight itemsets and 19 items, i.e., *size*($\mathbf{s}_1$)=8 and *len*($\mathbf{s}_1$)=19.

*Definition 2:* (Pattern with self-adaptive gaps [30]) A pattern with self-adaptive gaps is expressed as $\mathbf{p}$=$p_1 * p_2 * \cdots * p_o * \cdots * p_m$ or $\mathbf{p}$=$p_1p_2\cdots p_o\cdots p_m$, where $p_o(1 \le o \le m)$ is an itemset, and '$*$' represents any number of any itemsets.

*Definition 3:* (Occurrence and nonoverlapping occurrence [19]) Given a sequence $\mathbf{s}$=$s_1s_2\cdots s_n$ and a pattern $\mathbf{p}$=$p_1p_2\cdots p_m$, if $\mathbf{l}$=$< l_1, l_2, \ldots, l_m >$ satisfies $p_1 \subseteq s_{l_1}, p_2 \subseteq s_{l_2}, \ldots, p_i \subseteq s_{l_m}(1 \le l_1 < l_2 < \cdots < l_m \le n)$, then $\mathbf{l}$ is an occurrence of $\mathbf{p}$ in $\mathbf{s}$. We suppose there is another occurrence $\mathbf{l}'$=$< l'_1, l'_2, \ldots, l'_m >$. For any $1 \le q \le m$, if $l_q \ne l'_q$, then $\mathbf{l}$ and $\mathbf{l}'$ are two nonoverlapping occurrences.

*Definition 4:* (Support [19]) The support of pattern $\mathbf{p}$ in sequence $\mathbf{s}$ is the number of nonoverlapping occurrences, denoted by $sup(\mathbf{p}, \mathbf{s})$. The support of pattern $\mathbf{p}$ in sequential database $D$ is the sum of support of pattern $\mathbf{p}$ in each sequence $\mathbf{s}_i$, i.e., $sup(\mathbf{p}, D) = \sum_{i=1}^{k} sup(\mathbf{p}, \mathbf{s}_i)$.

*Example 3:* Given a sequence $\mathbf{s}_1 = (a)(abcd)(a)(abcd)(acd)(acd)(de)(d)$ and a pattern $\mathbf{p}$= $p_1p_2p_3 = (a)(b)(cd)$, all occurrences of $\mathbf{p}$ in $\mathbf{s}_1$ are $< 1, 2, 4 >$, $< 1, 2, 5 >$, $< 1, 2, 6 >$, $< 1, 4, 5 >$, $< 1, 4, 6 >$, $< 2, 4, 5 >$, $< 2, 4, 6 >$, $< 3, 4, 5 >$ and $< 3, 4, 6 >$, where $< 1, 2, 5 >$ and $< 1, 2, 6 >$ are two overlapping occurrences, since 1 and 2 are used in the same position in the two occurrences. $< 1, 2, 5 >$ and $< 2, 4, 6 >$ are two nonoverlapping occurrences, since 2 is used in different positions in the two occurrences. Therefore, the nonoverlapping occurrences of $\mathbf{p}$ in $\mathbf{s}_1$ are $< 1, 2, 5 >$ and $< 2, 4, 6 >$, and the support of $\mathbf{p}$ in $\mathbf{s}_1$ is $sup(\mathbf{p}, \mathbf{s}_1) = 2$. Similarly, $sup(\mathbf{p}, \mathbf{s}_2) = 3$ and $sup(\mathbf{p}, \mathbf{s}_3) = 0$. Hence, the support of $\mathbf{p}$ in $D$ is $sup(\mathbf{p}, D) = \sum_{i=1}^{3} sup(\mathbf{p}, \mathbf{s}_i) = 2 + 3 + 0 = 5$.

*Definition 5:* (Frequent pattern [19]) Given a minimum support threshold $minsup$, pattern $\mathbf{p}$ is frequent if its support in $D$ is not smaller than $minsup$, i.e., $sup(\mathbf{p}, D) \ge minsup$.

*Example 4:* We use the same pattern $\mathbf{p} = (a)(b)(cd)$ in Example 3 and $minsup$=4. Pattern $\mathbf{p} = (a)(b)(cd)$ is frequent, since $sup(\mathbf{p}, D) = 5 \ge minsup$.

TABLE II
Notations

| Symbol | Description |
|---|---|
| $D$ | Sequential database $D$ with $k$ sequences |
| $\mathbf{s}_i$ | A sequence in $D$ composed of multiple itemsets |
| $size(\mathbf{s}_i)$ | The size of sequence $\mathbf{s}_i$ |
| $len(\mathbf{s}_i)$ | The length of sequence $\mathbf{s}_i$ |
| $\mathbf{p}$ | A pattern with length $m$ |
| *sup*($\mathbf{p}$, $\mathbf{s}$) | The number of occurrences of $\mathbf{p}$ in $\mathbf{s}$ |
| *sup*($\mathbf{p}$, $D$) | The number of occurrences of $\mathbf{p}$ in $D$ |
| $minsup$ | The minimum support threshold |
| $mincf$ | The predefined confidence threshold |
| $\mathbf{p} \rightarrow \mathbf{r}$ | A co-occurrence rule |
| *conf*($\mathbf{p} \rightarrow \mathbf{r}$) | The confidence of co-occurrence rule $\mathbf{p} \rightarrow \mathbf{r}$ |

*Definition 6:* (S co-occurrence rule, I co-occurrence rule, rule antecedent, rule consequent, and confidence) Suppose we have patterns $\mathbf{p}$=$p_1p_2\cdots p_n$ and $\mathbf{q}$=$p_1p_2\cdots p_{n-1}q_nq_{n+1}\cdots q_m(n < m)$, where $p_n = g_{n1}g_{n2}\cdots g_{nk}$ and $q_n = g_{n1}g_{n2}\cdots g_{n(t-1)}g_{nt}$ $(1 \le k \le t)$. If $k$=$t$, $p_n$=$q_n$, *len*($p_n$)=*len*($q_n$), and $\mathbf{r}$=$q_{n+1}q_{n+2}\cdots q_m$, then $\mathbf{r}$ is an S co-occurrence pattern of $\mathbf{p}$ and $\mathbf{p}\rightarrow \mathbf{r}$ is an S co-occurrence rule. If $k < t$, $p_n \ne q_n$, *len*($p_n$) <*len*($q_n$), and $\mathbf{r} = (* g_{n(k+1)}\cdots g_{n(t-1)}g_{nt})q_{n+1}\cdots q_m$, then $\mathbf{r}$ is an I co-occurrence pattern of $\mathbf{p}$ and $\mathbf{p}\rightarrow \mathbf{r}$ is an I co-occurrence rule. Moreover, $\mathbf{p}$ and $\mathbf{r}$ are the antecedent and consequent of the co-occurrence rule, respectively. The ratio of the supports of patterns $\mathbf{q}$ and $\mathbf{p}$ is the confidence of rule $\mathbf{p}\rightarrow \mathbf{r}$, which means the probability that pattern $\mathbf{r}$ occurs when pattern $\mathbf{p}$ occurs, denoted by $conf(\mathbf{p} \rightarrow \mathbf{r}, D) = sup(\mathbf{q}, D)/sup(\mathbf{p}, D)$.

*Example 5:* Given a frequent pattern $\mathbf{p}$=$(a)(b)(c)$, if $\mathbf{q}$=$(a)(b)(c)(d)$, then $p_3 = q_3$, and $(a)(b)(c) \rightarrow (d)$ is an S co-occurrence rule with confidence 4/5 = 0.8. If $\mathbf{q}$=$(a)(b)(cd)$, then $p_3 \ne q_3$, and $(a)(b)(c) \rightarrow (*d)$ is an I co-occurrence rule with confidence 5/5 = 1.0.

*Definition 7:* (Strong co-occurrence rule) If $conf(\mathbf{p} \rightarrow \mathbf{r})$ is not smaller than a given minimum confidence threshold $mincf$, i.e., $conf(\mathbf{p} \rightarrow \mathbf{r}) \ge mincf$, then $\mathbf{p} \rightarrow \mathbf{r}$ is a strong co-occurrence rule.

*Example 6:* If $mincf$=1, then $(a)(b)(c) \rightarrow (d)$ is not a strong S co-occurrence rule, while $(a)(b)(c) \rightarrow (*d)$ is a strong I co-occurrence rule, since according to Example 5, their confidences are 0.8 and 1.0, respectively.

*Definition 8:* (CoNR mining) Given a sequential database $D$, a prefix pattern $\mathbf{p}$ and $mincf$, the aim of CoNR mining is to mine all strong S and I co-occurrence rules.

*Example 7:* Suppose we have $\mathbf{p}$=$(a)(b)(c)$ and $minsup$=5. In Example 3, the supports of $(a)(b)(cd)$ and $(a)(b)(c)(c)$ are both 5. From Definition 7, if $mincf$= 1, then $(a)(b)(c) \rightarrow (c)$ is a strong S co-occurrence rule and $(a)(b)(c) \rightarrow (*d)$ is a strong I co-occurrence rule. Thus, there are two CoNRs in $D$: $(a)(b)(c) \rightarrow (c)$ and $(a)(b)(c) \rightarrow (*d)$.

The symbols used in this paper are shown in Table II.

## IV. Proposed Algorithm

This section proposes a CoNR-Miner algorithm to mine strong S and I co-occurrence rules with a specific prefix pattern $\mathbf{p}$. The framework of CoNR-Miner is shown in Fig. 1.

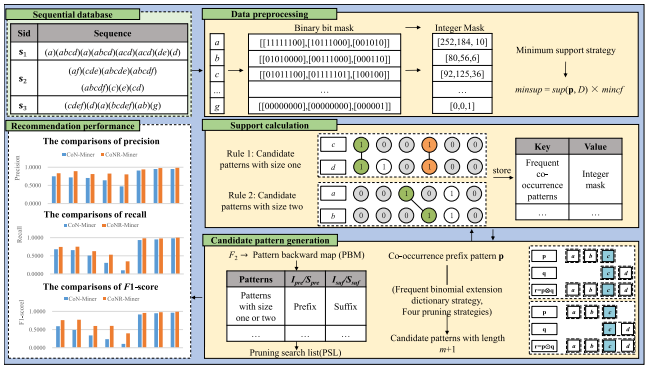



Fig. 1. Framework of CoNR-Miner. CoNR-Miner contains three parts: data preprocessing, support calculation, and candidate pattern generation. At data preprocessing stage, an integer mask dictionary is proposed and a minimum support strategy is employed to improve the efficiency. At support calculation stage, the support of a superpattern is calculated according to the integer mask dictionary of the subpattern. At candidate pattern generation stage, a frequent binomial extension dictionary strategy and four pruning strategies are proposed to reduce the candidate patterns.

### A. Data Preprocessing

This section proposes an integer mask dictionary to convert the sequential database into an integer mask and adopts a minimum support strategy [14] to calculate the minimum support $minsup$, i.e., $minsup = sup(\mathbf{p}, D) \times mincf$. An integer mask dictionary consists of keys and values. The key stores the item and the value stores its integer mask. The construction of an integer mask is as follows.

For item $x$ in sequence $\mathbf{s}_i$, we use a binary bit mask, denoted by $B_i(x) = [b_{i1}b_{i2}\dots b_{ir}]$, to represent its occurrences, where $b_{ij}(1 \le j \le r)$ is set to 1 if $x$ occurs in itemset $s_{ij}$, and $j$ is an occurrence position; otherwise, it is 0. Binary bit mask $B_i(x)$ is converted to integer mask $M_i(x)$, according to (1), to reduce space consumption.

$$M_i(x) = \sum_{j=1}^{r} b_{ij} \times 2^{r-j} \tag{1}$$

*Example 8:* Given sequence $\mathbf{s}_1 = (a)(abcd)(a)(abcd)(acd)(acd)(de)(d)$. Taking item '$b$' as an example, the first bit of '$b$' is 0, since it does not occur in $p_1$=$(a)$. The second bit of '$b$' is 1, since it occurs in $p_2 = (abcd)$. Similarly, the binary bit mask of $b$ is [01010000], and the occurrence positions are [2, 4]. According to (1), the binary bit mask of '$b$' is converted to an integer mask, i.e., $0 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 80$. Similarly, the binary bit masks of '$b$' in sequences $\mathbf{s}_2$ and $\mathbf{s}_3$ are [00111000] and [000110], respectively, and integer masks of them are 56 and 6, respectively. The binary bit mask and integer mask of each item are shown in Table III .

The pseudocode of processing the database as an integer mask dictionary (IMD) is given in Algorithm 1.

The IMD algorithm first initializes integer mask dictionary $M$ as an empty dictionary (Line 1). Next, it scans sequential database $D$ and reads each sequence, each itemset, and each item (Lines 2 to 4). For each item $e$, IMD updates $M[e][i]$ by setting the bit corresponding to the position of the current item (counting from the end of the sequence) and performing a bitwise OR operation with the existing mask (Line 5). Finally, IMD returns integer mask dictionary $M$ (Line 9).

TABLE III
BINARY BIT MASK AND INTEGER MASK OF DATABASE $D$

| Key | Binary bit mask | Integer mask |
|---|---|---|
| $a$ | [[11111100],[10111000],[001010]] | [252,184,10] |
| $b$ | [[01010000],[00111000],[000110]] | [80,56,6] |
| $c$ | [[01011100],[01111101],[100100]] | [92,125,36] |
| $d$ | [[01011111],[01111001],[110100]] | [95,121,52] |
| $e$ | [[00000010],[01100010],[100100]] | [2,98,36] |
| $f$ | [[00000000],[10011000],[100100]] | [0,152,36] |
| $g$ | [[00000000],[00000000],[000001]] | [0,0,1] |

**Algorithm 1:** IMD: Processing the Database as an Integer Mask Dictionary.

**Input:** Sequential database $D$
**Output:** Integer mask dictionary $M$
1: $M \leftarrow \{\}$;
2: **for** each sequence $\mathbf{s}_i$ in $D$ **do**
3: **for** each itemset $\mathbf{s}_{ij}$ in $\mathbf{s}_i$ **do**
4: **for** each item $e$ in $\mathbf{s}_{ij}$ **do**
5: $M[e][i] \leftarrow M[e][i] | (1 << ((len(\boldsymbol{s}_i) - 1) - j))$;
6: **end for**
7: **end for**
8: **end for**
9: **return** $M$;

### B. Support Calculation

To improve efficiency in support calculation, we propose a SupIM algorithm, based on the integer masks of its subpatterns, to calculate the support of a superpattern. SupIM has two rules.

**Rule 1:** For *size*(**p**)=1, the integer mask of **p** is obtained by bitwise AND of the integer masks of all items in **p**; the number of one in the integer mask of pattern **p** is its support.

**Rule 2:** For *size*(**p**)=$m(m \ge 2)$, we assume that patterns **p**=$p_1p_2\cdots p_m$, **c**=$p_1p_2\cdots p_{m-1}$, and **d**=$p_m$, and the integer masks of **c** and **d** are $mask_1$=$[c_1c_2\cdots c_g\cdots c_u]$ and $mask_2$=$[d_1d_2\cdots d_h\cdots d_u](c_g,\ d_h \in \{0,1\})$, respectively. If $d_h$ is the first unused position, both $d_h$ and $c_g$ are 1, and $h > g$, then $d_h$ is the matching position of $c_g$, and the number of one in the integer mask of pattern **p** is its support. The specific processes are as described below.

Gradually shift $mask_1$ bitwise to the right, use bitwise AND to calculate the matching bit with $mask_2$, use bitwise XOR to clear the matched bit between $mask_1$ and $mask_2$, and use bitwise OR to record the matching bit. Iterate the above processes, until no more matches can be found.

*Theorem 1:* The support calculation of pattern with size $m$ $(m \ge 2)$, i.e., Rule 2, is complete.

*Proof:* Suppose there are $k$ valid occurrences of **p** in the sequence. By the anti-monotonicity property, **c** and **d** each occur at least $k$ times. We denote the $k$ occurrences of **c** by $\alpha_1, \alpha_2, \dots, \alpha_k$ (where $\alpha_1 < \alpha_2 < \dots < \alpha_k$, all of which are bit indices of $mask_1$), and the corresponding occurrences of **p**

by $\beta_1, \beta_2, \ldots, \beta_k$ (where $\beta_1 < \beta_2 < \ldots < \beta_k$). It follows that **d** also occurs at least $k$ times at bit indices $\beta_1, \beta_2, \ldots, \beta_k$, (where $\beta_1 < \beta_2 < \ldots < \beta_k$, all of which are bit indices of $mask_2$), satisfying $\alpha_1 < \beta_1, \alpha_2 < \beta_2, \ldots, \alpha_k < \beta_k$.

Proof by contradiction. According to Rule 2, we assume that the algorithm can only generate $k-1$ occurrence positions. Thus, there must exist at least one pair of occurrence positions $(\alpha_a, \beta_b)$ not matched by the algorithm, where $\alpha_a \in \{\alpha_1, \ldots, \alpha_k\}$ and $\beta_b \in \{\beta_1, \ldots, \beta_k\}$. We will analyze in two cases.

Case 1: $\alpha_a < \beta_b$. Let $t = \beta_b - \alpha_a$ be the required right-shift distance. When $mask_1$ is right-shifted by $t$ bits, the valid bit corresponding to $\alpha_a$ moves to position $\alpha_a + t = \beta_b$. At the $t$-th iteration, the bitwise AND operation $(mask_1 \gg t)\&mask_2$ will output a 1 at position $\beta_b$, since both the shifted $mask_1$ and $mask_2$ have a 1 at this position. The matching bit is then recorded into the result $mask$ via the bitwise OR operation. The XOR operation only clears bits of already matched positions in $mask_1$. In Rule 2, each prefix bit in $mask_1$ can only be matched once and is then cleared by XOR. If $\alpha_a$ is cleared before the $t$-th iteration, this prefix position will be fully occupied and cannot form a real occurrence $(\alpha_a, \beta_b)$. Thus, $\alpha_a$ remains 1in $mask_1$ at the $t$-th iteration. The pair $(\alpha_a, \beta_b)$ should be detected and recorded, which contradicts the assumption that it is not matched.

Case 2: $\alpha_a \geq \beta_b$. Consider the first $b$ occurrences of ***d***: $\beta_1 < \beta_2 < \ldots < \beta_b$. By the property of pattern occurrence, there exist corresponding $b$ occurrences of ***c***: $\alpha_1 < \alpha_2 < \ldots < \alpha_b$, with $\alpha_b < \beta_b$. According to Rule 2, each of the previous $b-1$ occurrences $\beta_1, \ldots, \beta_{b-1}$ will be matched with an unused occurrence of ***c*** via the right-shift and AND operations, after which the corresponding bit in $mask_1$ is cleared by XOR. Thus, there must remain at least one unused occurrence $\alpha_a$ in $\{\alpha_1, \ldots, \alpha_b\}$ such that $\alpha_a \leq \alpha_b$. Since $\alpha_b < \beta_b$, we have $\alpha_a < \beta_b$, which directly contradicts $\alpha_a \geq \beta_b$.

Combining both cases, Rule 2 is complete.

To reduce the number of iterations in Rule 2, we propose an early termination strategy.

**Early termination strategy**: If the effective bit length of $mask_1$ after shifting is less than the current effective bit length of $mask_2$, i.e., $mask_1$.bit_length()< ($mask_2$ AND NOT $mask_2$).bit_length(), then terminate.

*Theorem 2:* Early termination strategy is complete.

*Proof:* If, after a right shift, the position of the highest bit in $mask_1$ is lower than that of the lowest bit in $mask_2$, then all bits in $mask_1$ precede all bits in $mask_2$. This makes a match according to Rule 2 impossible. Any further right shifts of $mask_1$ can only be below the position of its highest bit, such that no future matches will occur. Hence, the early termination strategy is complete.

Example 9 illustrates the principle of SupIM.

*Example 9:* We use $\mathbf{s}_3 = (cdef)(d)(a)(bcdef)(ab)(g)$ from Table I and patterns **t**=$(cd)$ and **p**=$(a)(b)$ to illustrate the principles of SupIM.

1) Since *size*(**t**)=1, we calculate the support of **t** according to Rule 1. The binary bit masks for items $c$ and $d$ are [100100] and [110100], and the corresponding integer masks are 36 and 52, respectively. Hence, the value by bitwise AND is [100100], and its integer mask is 36. Hence, the support of **t** in $\mathbf{s}_3$ is two, since one occurs twice in [100100].

2) Since *size*(**p**)=2, we calculate the support of **p** according to Rule 2. The binary bit masks for items $a$ and $b$ are $mask_1$=[001010] and $mask_2$=[000110], and the corresponding integer masks are 10 and 6, respectively. Shift [001010] one bit to the right and obtain $mask_1$ =[000101]. Use bitwise AND with [000110] to calculate the matching bit, which is [000100], and use bitwise XOR to clear the matched bits. Currently, $mask_1$=[000001] and $mask_2$=[000010]. Use bitwise OR operation to record the matching bit [000100]. However, the effective bit length of $mask_1$ is 1 and the effective bit length of $mask_2$ is 2. This meets the early termination strategy, thus the matching process ends. The binary value of the binary bit mask of pattern **p** is [000100] and the corresponding integer mask is 4. Hence, the support of **p** in $\mathbf{s}_3$ is one, since one occurs once in [000100].

Pseudocode for SupIM is given in Algorithm 2.

The SupIM algorithm first initializes $sup(\mathbf{p}, D)$ to zero and sets $Num$ to the number of sequences in $D$ (Line 1). It then defines **c** as the prefix of **p** and **e** as the last itemset, and allocates $M$[**p**] as an empty list (Line 2). If the size of **p** is one, according to Rule 1 (Lines 3–7), for each sequence it sets $M$[**p**][$t$] to the bitwise AND of $M$[**c**][$t$] and $M$[**e**][$t$], and adds support to $sup(\mathbf{p}, D)$. Otherwise, according to Rule 2 (Lines 8–26), for each sequence it computes $mask_1 = M[\mathbf{c}][t] >> 1$ and $mask_2 = M[\mathbf{e}][t]$, then iteratively finds common bits to build a temporal mask with an early termination strategy (Lines 18–20), stores the mask into $M$[**p**][$t$], and again adds its support. Finally, SupIM returns $sup(\mathbf{p}, D)$(Line 28).

### C. Candidate Pattern Generation

Classical sequential pattern mining methods [53] employ I-extension and S-extension to generate candidate patterns. I-extension adds item $h$ to the last itemset of **p**, denoted by $\mathbf{p} \odot h$, and S-extension adds item $h$ as a new itemset after the last itemset of **p**, denoted by $\mathbf{p} \otimes h$. Some improvement methods add frequent item $h$ to **p**. However, these methods generate many redundant candidate patterns. To reduce candidate patterns, we propose a frequent binomial extension dictionary (FBED) strategy. More importantly, we propose four pruning strategies to further prune candidate patterns, based on pattern backward map (PBM) [44]. PBM is constructed from frequent patterns with length two. The size of the frequent patterns with length two can be one or two. If it is one, we use $I_{pre}$ and $I_{suf}$ to store the prefix and suffix of each frequent pattern, respectively; if it is two, we use $S_{pre}$ and $S_{suf}$ to store the prefix and suffix of each frequent pattern, respectively.

*Example 10:* We assume that $minsup$=5. The frequent patterns with size one and length two in $D$ are $(ab), (ac), (ad), (bc), (bd), (cd)$, and $(de)$, as shown in Table IV; the frequent patterns with size two and length two are $(a)(a), (a)(b), (a)(c), (a)(d), (b)(a), (b)(c), (b)(d), (c)(a), (c)(b), (c)(c), (c)(d), (d)(a), (d)(b), (d)(c), (d)(d)$, and $(e)(d)$, as shown in Table V.

**Algorithm 2:** SupIM: Calculating the Support of a Pattern.

**Input:** Sequential database $D$, pattern $\mathbf{p}$, and integer mask dictionary $M$
**Output:** sup$(\mathbf{p}, D)$

```
sup(p, D) ← 0; Num ← the number of sequences in database D;
c ← p[: −1]; e ← p[−1]; M[p] ← [] ∗ Num;
if size(p) == 1 then
  for t = 1 to Num do
    M[p][t] ← M[c][t] AND M[e][t];
    sup(p, D) ← sup(p, D)+ the number of one in mask;
  end for
else
  for t = 1 to Num do
    list1 ← M[c][t]; list2 ← M[e][t]; mask ← 0;
    mask1 ← list1 >> 1; mask2 ← list2;
    while mask1 > 0 and mask2 > 0 do
      masktemp ← mask1 AND mask2;
      if masktemp ≠ 0 then
        mask1 ← mask1 XOR masktemp;
        mask2 ← mask2 XOR masktemp;
        mask ← mask OR masktemp;
        if mask1.bit_length()< (mask2 AND NOT mask2).bit_length() then
          break;
        end if
        mask1 >>= 1;
      end if
    end while
    M[p][t] ← mask;
    sup(p, D) ← sup(p, D)+ the number of one in mask;
  end for
end if
return sup(p, D);
```

TABLE IV
ALL FREQUENT PATTERNS WITH SIZE ONE AND LENGTH TWO

| Patterns | $I_{pre}$ | $I_{suf}$ |
|---|---|---|
| $(ab), (ac), (ad)$ | $a$ | $[b, c, d]$ |
| $(bc), (bd)$ | $b$ | $[c, d]$ |
| $(cd)$ | $c$ | $[d]$ |
| $(de)$ | $d$ | $[e]$ |

To reduce the number of candidate patterns, we propose a FBED strategy based on PBM.

**FBED strategy**: Assuming we join prefix pattern $\mathbf{p} = p_1p_2\cdots p_m(p_m = i_{m1}i_{m2}\cdots i_{mt})$ with a frequent binomial pattern $\mathbf{q}$, the candidate pattern generation can be divided into two cases:

**Case 1:** When *size*($\mathbf{q}$)=1, we assume that $\mathbf{q} = q_1 = (e_1e_2)$ and $i_{mt} = e_1$. If $e_2 \in I_{\text{suf}}[e_1]$, then we generate a superpattern with size *m*: $\mathbf{r} = \mathbf{p} \odot e_2 = p_1p_2\cdots p_{m-1}(i_{m1}i_{m2}\cdots i_{mt}e_2)$ to make an I-extension.

TABLE V
ALL FREQUENT PATTERNS WITH SIZE TWO AND LENGTH TWO

| Patterns | $S_{pre}$ | $S_{suf}$ |
|---|---|---|
| $(a)(a), (a)(b), (a)(c), (a)(d)$ | $a$ | $[a, b, c, d]$ |
| $(b)(a), (b)(c), (b)(d)$ | $b$ | $[a, c, d]$ |
| $(c)(a), (c)(b), (c)(c), (c)(d)$ | $c$ | $[a, b, c, d]$ |
| $(d)(a), (d)(b), (d)(c), (d)(d)$ | $d$ | $[a, b, c, d]$ |
| $(e)(d)$ | $e$ | $[d]$ |

**Case 2:** When *size*($\mathbf{q}$)=2, we assume that $\mathbf{q} = q_1q_2 = (e_1)(e_2)$ and $i_{mt} = e_1$. If $e_2 \in S_{\text{suf}}[e_1]$, then we generate a superpattern with size *m*+1: $\mathbf{r} = \mathbf{p} \otimes e_2 = p_1p_2\cdots p_m(e_2)$ to make an S-extension.

Example 11 illustrates the principle of the FBED strategy.

*Example 11:* Assume a pattern $\mathbf{p}=(a)(b)(c)$ and $minsup = 5$. Thus, $i_{mt} = c$. From Example 10, $I_{suf}[c] = [d]$, and the candidate pattern generated by I-extension is $(a)(b)(cd)$, according to Case 1. Moreover, $S_{suf}[c] = [a, b, c, d]$, and the candidate patterns generated by S-extension are $(a)(b)(c)(a), (a)(b)(c)(b), (a)(b)(c)(c)$, and $(a)(b)(c)(d)$, according to Case 2.

To prune unpromising candidate patterns, we propose a pruning search list.

*Definition 9:* (Pruning search list, PSL) We assume the prefix pattern $\mathbf{p} = p_1p_2\cdots p_j\cdots p_m(p_1 = g_{11}g_{12}\cdots g_{1t_1}, p_2 = g_{21}g_{22}\cdots g_{2t_2}, \ldots, p_j = g_{j1}g_{j2}\cdots g_{jt_j}, \ldots, p_m = g_{m1}g_{m2}\cdots g_{mt_m})$, where $1 \le j \le m$, and the concatenation set of all the items of $p_j$ is denoted by $US(p_j) = g_{j1} \cup g_{j2} \cup \cdots \cup g_{jt_j} = \{i_1, i_2, \ldots, i_k, \ldots, i_z\}$ $(1 \le k \le z \le t_j;$ $\forall m, n \in \{1, 2, \ldots, z\}, m \ne n \Rightarrow i_m \ne i_n)$. For I-extension, we design I-List, denoted by $IL[p_j] = [I_1 \cap I_2 \cap \cdots \cap I_k \cap \cdots \cap I_z]$, where $I_k = I_{\text{suf}}[i_k]$ and $I_k \in US(p_j)$. Similarly, for S-extension, we design S-List, denoted by $SL[p_j] = [S_1 \cap S_2 \cap \cdots \cap S_k \cap \cdots \cap S_z]$, where $S_k = S_{\text{suf}}[i_k]$ and $S_k \in US(p_j)$. There are two cases for calculating the PSL.

**Case 1:** If *size*($\mathbf{p}$)=1 and all items in $\mathbf{p}$ are $T = US(\mathbf{p}) = US(p_1)$, then we store $IL[\mathbf{p}]$ in *oneIlist* and $SL[\mathbf{p}]$ in *Slist*.

**Case 2:** If *size*($\mathbf{p}$)>1, then we create *Slist*, *mSlist*, and *mIlist*, as shown below.

(1) For $US(\mathbf{p})$, we store *SL*[$\mathbf{p}$] in *Slist*.

(2) For $T_1 = US(p_1p_2\cdots p_{m-1})$, we store $SL[p_1p_2\cdots p_{m-1}]$ in *mSlist*.

(3) For $T_2 = US(p_m)$, we store $IL[p_m]$ in *mIlist*.

*Example 12:* We use $(abc)$ and $(a)(b)(c)$ to illustrate the principles of PSL.

**1.** For $\mathbf{p}=(abc)$, since *size*($\mathbf{p}$)=1, we generate PSL according to Case 1.

All items in $\mathbf{p}$ are $T = US(\mathbf{p}) = \{a, b, c\}$. From Example 10, $I_1 = I_{\text{suf}}[a] = [b, c, d]$, $I_2 = I_{\text{suf}}[b] = [c, d]$, and $I_3 = I_{\text{suf}}[c] = [d]$. Thus, $IL[\mathbf{p}] = [b, c, d] \cap [c, d] \cap [d] = [d]$ is stored in oneIlist=$[d]$. Similarly, $S_1 = S_{\text{suf}}[a] = [a, b, c, d]$, $S_2 = S_{\text{suf}}[b] = [a, c, d]$, and $S_3 = S_{\text{suf}}[c] = [a, b, c, d]$. Thus, $SL[\mathbf{p}] = [a, b, c, d] \cap [a, c, d] \cap [a, b, c, d] = [a, c, d]$ is stored in *Slist*=$[a, c, d]$. Thus, compared with Example 11, *Slist*=$[a, c, d]$ is less than $S_{suf}[c] = [a, b, c, d]$.

**2.** For $\mathbf{p}=(a)(b)(c)$, since *size*(**p**)=3>1, we generate PSL according to Case 2.

(1) For $US(\mathbf{p})=\{a,b,c\}$, from Example 12, we store $SL[\mathbf{p}]=[a,c,d]$ in *Slist*.

(2) For $T_1=US(p_1p_2)=\{a,b\}$. From Example 10, $S_1=S_{\text{suf}}[a]=[a,b,c,d]$, and $S_2=S_{\text{suf}}[b]=[a,c,d]$. Thus, $SL[p_1p_2]=[a,b,c,d]\cap[a,c,d]=[a,c,d]$, i.e., we store $SL[p_1p_2]=[a,c,d]$ in *mSlist*.

(3) For $T_2=US(p_3)=\{c\}$. From Example 10, $I_1=I_{\text{suf}}[c]=[d]$. Thus, $IL[p_3]=[d]$, i.e., we store $IL[p_3]=[d]$ in *mIlist*.

Based on PSL, we propose four pruning strategies to further reduce the candidate patterns.

**Pruning strategy 1 (I-extension pruning strategy 1)**: If *size*(**r**)=*size*(**p**)=1 and $e_2\notin oneIlist$, then pattern **r** can be pruned.

*Theorem 3:* I-extension pruning strategy 1 is correct.

*Proof:* We suppose *size*(**r**)=*size*(**p**)=1. If $g_{1h}\in T$ $(1\le h\le t_1)$ and $e_2\notin oneIlist$, then $(g_{1h}e_2)$ is not a frequent pattern. According to the anti-monotonicity property, pattern **r** is not a frequent pattern either, since pattern **r** is a superpattern of $(g_{1h}e_2)$. Hence, pattern **r** can be pruned.

*Example 13:* From Example 12, **p**=$(abc)$, *oneIlist*=$[d]$. Suppose patterns $(abcd)$ and $(de)$ can generate the candidate pattern **r**=$(abcde)$. However, *size*(**r**)=*size*(**p**)=1 and $e_2$=$e\notin$*oneIlist*=$[d]$. Hence, pattern **r**=$(abcde)$ is not frequent, since $(ae)$, $(be)$, and $(ce)$ are not frequent. Hence, **r** can be pruned.

**Pruning strategy 2 (I-extension pruning strategy 2)**: If *size*(**p**)>1, *size*(**r**)=*size*(**p**), and $e_2\notin$*mIlist* or $e_2\notin$*mSlist*, then pattern **r** can be pruned.

*Theorem 4:* I-extension pruning strategy 2 is correct.

*Proof:* We suppose *size*(**p**)>1 and *size*(**r**)=*size*(**p**). Therefore, *size*(**r**)>1. If $g_{fh}\in T_1$ or $g_{fh}\in T_2$ $(1\le f\le m,\ 1\le h\le t_m)$, $e_2\notin$*mIlist* or $e_2\notin$*mSlist*, then $(g_{fh}e_2)$ or $(g_{fh})(e_2)$ is not a frequent pattern. According to the anti-monotonicity property, pattern **r** is not a frequent pattern, since pattern **r** is a superpattern of $(g_{fh}e_2)$ or $(g_{fh})(e_2)$. Hence, pattern **r** can be pruned.

*Example 14:* From Example 12, we know that **p**=$(a)(b)(c)$, *mIlist*=$[d]$ and *mSlist*=$[a,c,d]$. Patterns $(a)(b)(cd)$ and $(de)$ can generate candidate pattern **r**=$(a)(b)(cde)$. Thus, *size*(**p**)>1, *size*(**r**)=*size*(**p**), $e_2$=$e\notin$*mIlist*=$[d]$, and $e_2$=$e\notin$*mSlist*=$[a,c,d]$. In this case, pattern **r** =$(a)(b)(cde)$ is not frequent, since $(a)(e)$, $(b)(e)$, and $(ce)$ are not frequent. Hence, pattern **r** can be pruned.

Note that pruning strategies 1 and 2 cannot be applied in single sequences with items, since $I_{suf}$ is NULL. Similarly, if the average itemset length is very small and close to 1, then there are fewer I-extension operations. Thus, there are fewer chances to use pruning strategies 1 and 2.

**Pruning strategy 3 (S-extension pruning strategy):** If *size*(**r**)>*size*(**p**) and $e_2\notin$*Slist*, then pattern **r** can be pruned.

*Theorem 5:* S-extension pruning strategy is correct.

*Proof:* We assume that *size*(**r**)>*size*(**p**). Therefore, *size*(**r**)>1. If $g_{fh}\in T(1\le f\le m,\ 1\le h\le t_m)$ and $e_2\notin$*Slist*, then $(g_{fh})(e_2)$ is not a frequent pattern. According to the anti-monotonicity property, pattern **r** is not frequent, since pattern **r** is a superpattern of $(g_{fh})(e_2)$. Hence, pattern **r** can be pruned.

*Example 15:* From Example 12, **p**=$(a)(b)(c)$, *Slist*=$[a,c,d]$. Suppose we have patterns $(a)(b)(c)$ and $(c)(b)$, which can generate a candidate pattern **r**=$(a)(b)(c)(b)$. Thus, *size*(**r**)>*size*(**p**) and $e_2$=$b\notin$*Slist*=$[a,c,d]$. In this case, pattern **r**=$(a)(b)(c)(b)$ is not frequent, since $(b)(b)$ is not frequent. Hence, pattern **r** can be pruned.

**Pruning strategy 4 ( Superpattern pruning strategy )** : If pattern **r** is a superpattern of an infrequent pattern, then pattern **r** can be pruned.

*Theorem 6:* The superpattern pruning strategy is correct.

*Proof:* If pattern **r** is a superpattern of pattern **h**, then *sup*(**r**,*D*)≤*sup*(**h**,*D*), according to the anti-monotonicity property. If **h** is infrequent, then *sup*(**h**,*D*)< $minsup$. Thus, *sup*(**r**,*D*)< $minsup$ and **r** is infrequent and can be pruned.

Note that pruning strategies 1, 2, and 3 are prepruning stages for pruning strategy 4, reducing the number of judgments and improving the efficiency of candidate pattern generation.

In traditional mining methods, once a candidate pattern is an infrequent pattern it is discarded. However, to better utilize pruning strategy 4, we store all these infrequent patterns in set *I*. It is a time-consuming task to determine whether candidate pattern **r** is a superpattern of the patterns in set *I*. To quickly determine that **r** is not a superpattern of **h**, we propose two fast judgement strategies based on checking pattern and checking item, where **h** is a pattern in *I*.

*Definition 10:* (Checking pattern and Checking item) If we assume a prefix pattern $\mathbf{p}=p_1p_2\cdots p_m$ and a pattern $\mathbf{r}=p_1p_2\cdots p_m\cdots p_n$, then $\mathbf{c_r}=p_m\cdots p_n$ is the checking pattern of pattern **r** and the checking items are the intersection of all items of the checking pattern $\mathbf{c_r}$.

**Size judgement strategy**: If *size*($\mathbf{c_r}$)<*size*($\mathbf{c_h}$), then **r** is not a superpattern of **h**.

**E lemental judgement strate gy**: If some checking items of $\mathbf{c_h}$ do not belong to the checking items of $\mathbf{c_r}$, then **r** is not a superpattern of **h**.

*Theorem 7:* Size and elemental judgement strategies are correct.

*Proof:* If **r** is a superpattern of **h**, then *size*(**r**)≥*size*(**h**). However, *size*($\mathbf{c_r}$)<*size*($\mathbf{c_h}$). Thus, **r** is not a superpattern of **h**. Moreover, if **r** is a superpattern of **h**, then all checking items of $\mathbf{c_h}$ belong to the checking items of $\mathbf{c_r}$. Now, some checking items of $\mathbf{c_h}$ do not belong to the checking items of $\mathbf{c_r}$. Hence, **r** is not a superpattern of **h**.

Example 16 illustrates the principle of the two fast judgement strategies.

*Example 16:* We assume the prefix pattern **p**=$(a)(b)(c)$ and the infrequent pattern **h** = $(a)(b)(c)(a)$, $\mathbf{c_h}$=$(c)(a)$ and the checking items are $[a,c]$, according to Definition 10.

(1) If superpattern **r**=$(a)(b)(cd)$, then $\mathbf{c_r}$ =$(cd)$, *size*($\mathbf{c_r}$)=1<*size*($\mathbf{c_h}$)=2. According to size judgement strategy, $(a)(b)(cd)$ is not a superpattern of $(a)(b)(c)(a)$.

(2) If superpattern **r**=$(a)(b)(cd)(d)$, then $\mathbf{c_r}$ =$(cd)(d)$ and *size*($\mathbf{c_r}$)=*size*($\mathbf{c_h}$)=2. However, the checking items of $\mathbf{c_h}$ are $[a,c]$, which do not belong to the checking items of $\mathbf{c_r}$, which are

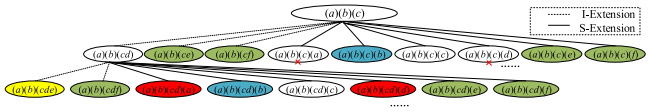



Fig. 2. Candidate pattern tree generated by I-extension and S-extension for $\mathbf{p}$=$(a)(b)(c)$. All nodes with colors are pruned by our strategies.

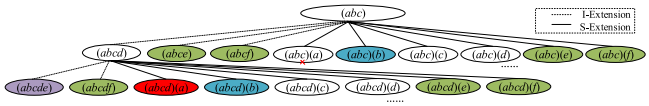



Fig. 3. Candidate pattern tree generated by I-extension and S-extension for $\mathbf{p}$=$(abc)$. All nodes with colors are pruned by our strategies.

$[c, d]$. According to elemental judgement strategy, $(a)(b)(cd)(d)$ is not a superpattern of $(a)(b)(c)(a)$.

Example 17 illustrates the principles of FBED and four pruning strategies.

*Example 17:* We assume $\mathbf{p}$=$(a)(b)(c)$. A candidate pattern tree generated by I-extension and S-extension for $\mathbf{p}$ is shown in Fig. 2. Compared with the enumeration strategy, pattern $(a)(b)(ce)$ and other green nodes are pruned by the FBED strategy. Pattern $(a)(b)(cde)$ is pruned by the I-extension pruning strategy 2 (Pruning strategy 2), shown as yellow nodes, and patterns $(a)(b)(c)(b)$ and $(a)(b)(cd)(b)$ are pruned by the S-extension pruning strategy (Pruning strategy 3), shown as blue nodes. Patterns $(a)(b)(cd)(a)$ and $(a)(b)(cd)(d)$ are superpatterns of $(a)(b)(c)(a)$ and $(a)(b)(c)(d)$, respectively. Hence, after knowing that $(a)(b)(c)(a)$ and $(a)(b)(c)(d)$ are infrequent (marked with a red "x"), $(a)(b)(cd)(a)$ and $(a)(b)(cd)(d)$ can be pruned by a superpattern pruning strategy (Pruning strategy 4), shown as red nodes.

Assuming that $\mathbf{p}$=$(abc)$, the candidate pattern tree generated by I-extension and S-extension for $\mathbf{p}$=$(abc)$ is shown in Fig. 3. Pattern $(abce)$ and other green nodes are pruned by the FBED strategy. Pattern $(abcde)$ is pruned by I-extension pruning strategy 1 (Pruning strategy 1), shown as purple nodes, and patterns $(abc)(b)$ and $(abcd)(b)$ are pruned by the S-extension pruning strategy (Pruning strategy 3), shown as blue nodes. $(abcd)(a)$ is a superpattern of $(abc)(a)$. Hence, after knowing that $(abc)(a)$ is infrequent, pattern $(abcd)(a)$ is pruned by the superpattern pruning strategy (Pruning strategy 4), shown as a red node.

We propose the FBEDFour algorithm, which uses FBED to generate candidate patterns and then uses the four pruning strategies to prune the candidate patterns shown in Algorithm 3.

The FBEDFour algorithm initializes $C_{i+1}$ as an empty set (Line 1). For each pattern $\mathbf{t}$ in $CF_i$, if the last item of $\mathbf{t}$ belongs to $I_{pre}$, it iterates over each item $e$ in $I_{suf}$[$\mathbf{t}$[-1][-1]]; if pruning strategies 1, 2, 3 or 4 is not satisfied, I-extension extends $\mathbf{t}$ with item $e$ to generate $\mathbf{r}$ and adds it to $C_{i+1}$ (Lines 2–11). Then, if the last item of $\mathbf{t}$ belongs to $S_{pre}$, it iterates over each item $e$ in $S_{suf}$[$\mathbf{t}$[-1][-1]]; if pruning strategy 3 or 4 is not satisfied, S-extension extends $\mathbf{t}$ with item $e$ to generate $\mathbf{r}$ and adds it to $C_{i+1}$ (Lines 12–21). Finally, it returns $C_{i+1}$ (Line 22).

**Algorithm 3:** FBEDFour: Generating Candidate Patterns Using FBED and Four Pruning Strategies:

**Input:** Co-occurrence frequent pattern sets $CF_i$, $PBM$
**Output:** Candidate pattern sets $C_{i+1}$

```
1:  C_{i+1} ← {};
2:  for each pattern t in CF_i do
3:    if t[-1][-1] in I_pre then
4:      for each item e in I_suf[t[-1][-1]] do
5:        if Pruning strategy 1, 2, 3 or 4 is satisfied then
6:          continue;
7:        end if
8:        r ←t⊙e;
9:        C_{i+1} ← C_{i+1}∪r;
10:     end for
11:   end if
12:   if t[-1][-1] in S_pre then
13:     for each item e in S_suf[t[-1][-1]] do
14:       if Pruning strategy 3 or 4 is satisfied then
15:         continue;
16:       end if
17:       r ←t⊗e;
18:       C_{i+1} ← C_{i+1}∪r;
19:     end for
20:   end if
21: end for
22: return C_{i+1};
```

### D. CoNR-Miner

The CoNR-Miner algorithm consists of the following steps.

**Step 1:** Convert sequential database $D$ into an integer mask $M$ using the IMD algorithm.

**Step 2:** Store frequent patterns with length one in $FP_1$.

**Step 3:** Calculate the supports of patterns with length two generated by $FP_1$ through I-extension and S-extension, store the frequent patterns in $FP_2$, and generate *PBM* and *PSL*.

**Step 4:** Generate frequent patterns with length greater than two and size one and store all frequent patterns in their integer masks.

**Step 5:** Use the FBED strategy to generate candidate pattern set $C_{i+1}$, according to $CF_i$, and use four pruning strategies to prune the candidate patterns, where if $i$=1, then $CF_1$ is the co-occurrence prefix pattern.

**Step 6:** For each pattern $\mathbf{p}$ in candidate pattern set $C_{i+1}$, calculate its support using the SupIM algorithm. Store the frequent patterns in the co-occurrence frequent pattern set $CF_{i+1}$.

**Step 7:** Iterate Steps 5 and 6 until the co-occurrence frequent pattern set $CF_i$ is empty. The co-occurrence sequential rules (CoNRs) are $CF_2 \cup CF_3 \cup \cdots CF_i$.

The CoNR-Miner algorithm is used to mine all CoNRs, as shown in Algorithm 4.

The CoNR-Miner algorithm first calls the IMD algorithm to preprocess sequential database $D$ and obtain integer mask dictionary $M$ (Line 1). Then, it stores all frequent patterns with length one in $FP_1$ (Line 2), and mines frequent patterns with length two generated by $FP_1$ and stores them in $FP_2$, and constructs $PBM$ and $PSL$ (Line 3). CoNR-Miner initializes $i$ to one, sets $CF_1$ to contain the given co-occurrence prefix frequent pattern $\mathbf{p}$,

**Algorithm 4:** CoNR-Miner: Mining all CoNRs.

**Input:** Sequential database $D$, co-occurrence prefix frequent pattern **p**, and $mincf$
**Output:** Co-occurrence sequential rule sets $CoNRs$

1: $M \leftarrow IMD(D)$;
2: Store frequent patterns with length one in $FP_1$;
3: Mining frequent patterns with length two generated by $FP_1$ and store them in $FP_2$, and construct $PBM$ and $PSL$;
4: $i \leftarrow 1$; $CF_1 \leftarrow$**p**; $CF \leftarrow \{\}$; $minsup =$SupIM(**p**, $D$) $\times$ $mincf$;
5: **while** $CF_i \neq NULL$ **do**
6: $C_{i+1} \leftarrow FBEDFour(CF_i)$; $CF_{i+1} \leftarrow \{\}$;
7: **for** each pattern **c** in $C_{i+1}$ **do**
8: $sup$(**c**, $D$) $\leftarrow$SupIM(**c**,$D$);
9: **if** $sup$(**c**, $D$) $\geq minsup$ **then**
10: $CF_{i+1} \leftarrow CF_{i+1} \cup$**c**;
11: **end if**
12: **end for**
13: $CF \leftarrow CF \cup CF_{i+1}$;
14: $i \leftarrow i + 1$;
15: **end while**
16: $CoNRs \leftarrow CF$;
17: **return** $CoNRs$;

initializes $CF$ as an empty set and calculates $minsup$ (Line 4). When $CF_i$ is not empty (Line 5), it generates candidate patterns $C_{i+1}$ by calling FBEDFour($CF_i$) and initializes $CF_{i+1}$ as an empty set (Line 6). For each candidate pattern **c** in $C_{i+1}$ (Line 7), CoNR-Miner calculates its support $sup$(**c**,$D$) using the supIM algorithm (Line 8); if the support is not smaller than $minsup$, it adds **c** to $CF_{i+1}$ (Lines 9–11). After processing all candidate patterns, it merges $CF_{i+1}$ into $CF$, increments $i$ by one (Lines 13–14), and iterates the process. Finally, CoNR-Miner returns $CF$ as the set of CoNRs (Lines 16–17).

### E. Space and Time Complexity Analysis

*Theorem 8:* The space complexity of CoNR-Miner is $O((s + n) \times (n + t) + t)$, where $s$, $n$, and $t$ are the numbers of sequences in $D$, items, and CoNRs, respectively.

*Proof:* CoNR-Miner has three parts: data preprocessing, support calculation and candidate pattern generation. In data preprocessing, each item corresponds to $s$ integer masks, resulting in a space complexity of $O(s \times n)$ for $n$ items. In support calculation, storing the integer masks of all CoNRs requires $O(t \times s)$ space. In candidate pattern generation, both the PBM and PSL structures are stored, each occupying $O(n^2)$ space. Storing the candidate pattern set, where each pattern from $CF_i$ is concatenated with $n$ suffix items, incurs a space complexity of $O(t \times n)$; storing the frequent patterns set, where $CF$ contains $t$ patterns, incurs a space complexity of $O(t)$. Therefore, the space complexity of CoNR-Miner is $O(s \times n + t \times s + n^2 + t \times n + t) = O((s + n) \times (n + t) + t)$.

*Theorem 9:* The time complexity of CoNR-Miner is $O(L + N \times S + t \times n)$, where $N$, $L$, and $S$ are the number of candidate patterns, the total length of $D$, and the size of all itemsets in $D$, respectively.

*Proof:* In data preprocessing, we need to scan the sequential database once, converting it into an integer mask dictionary. Hence, the time complexity of data preprocessing is $O(L)$. In support calculation, it requires traversing $S$ integer masks. For all candidate patterns, the time complexity of SupIM is $O(N \times S)$. In candidate pattern generation, iterate through all $t$ patterns in $CF_i$, traversing $n$ items in each pattern. Thus, the time complexity for candidate pattern generation is $O(t \times n)$. Hence, the time complexity of CoNR-Miner is $O(L + N \times S + t \times n)$.

TABLE VI
DESCRIPTION OF DATABASES

| Database | Name | $\|\Sigma\|$ | $\|s\|$ | $\|I\|$ | $\|L\|$ | $Avg(i)$ |
|---|---|---|---|---|---|---|
| SDB1 | Babysale | 6 | 359 | 3,966 | 10,013 | 2.520 |
| SDB2 | E-shop | 283 | 766 | 5,258 | 47,322 | 9.000 |
| SDB3 | Movie | 18 | 91 | 505 | 4,689 | 9.290 |
| SDB4 | PM2.5 | 582 | 60 | 1,789 | 34,743 | 19.420 |
| SDB5 | Game-sale | 12 | 38 | 1,534 | 5,903 | 3.850 |
| SDB6 | Chicago-Crime | 29 | 1,675 | 16,750 | 34,574 | 2.064 |
| SDB7 | Credit | 8 | 12,760 | 765,600 | 765,600 | 1.000 |
| SDB8 | Letter | 10 | 10,000 | 180,000 | 180,000 | 1.000 |

Note 1: SDB1 is a sale database for baby products, and can be downloaded from https://tianchi.aliyun.com/dataset/45.
Note 2: SDB2 is a click-stream database for online shopping, and can be downloaded from https://philippe-fournier-viger.com/spmf/datasets/e_shop.txt.
Note 3: SDB3 is a database of user viewing ratings collected by MovieLens, and can be downloaded from http://grouplens.org/datasets/.
Note 4: SDB4 is a database containing PM2.5 data from the US Embassy in Beijing, and can be downloaded from https://archive.ics.uci.edu/ml/datasets/Beijing+PM2.5+Data.
Note 5: SDB5 is a database of video game sales in the United States, and can be downloaded from http://www.kaggle.com/rush4ratio/video-game-sales-with-ratings.
Note 6: SDB6 is a crime database from Chicago, and can be downloaded from http://philippe-fournierviger.com/spmf/datasets/Chicago Crimes 2001 to 2017 utility.txt.
Note 7: SDB7 is a credit card transaction database, and can be downloaded from http://www.kesci.cpm/mw/dataset/5b56a592fc7e9000103c0442.
Note 8: SDB8 is a real database, and can be downloaded from http://archive.ics.uci.edu/ml.

## V. EXPERIMENTAL RESULTS AND ANALYSIS

### A. Databases and Baseline Algorithms

Eight databases are selected and are described in Table VI. $|\Sigma|$ is the number of items, $|s|$ is the number of sequences, $|I|$ is the number of itemsets, $|L|$ is the total length of all sequences, and $Avg(i)$ is the average length of each itemset.

To validate the performance of CoNR-Miner, 16 competitive algorithms are selected.

1) CoNR-Invert: To verify the performance of the integer mask dictionary in support calculation, we design CoNR-Invert, which uses a position index dictionary to store the occurrence positions of patterns.
2) DFOM-CoNR [54] and Pro-CoNR [7]: To evaluate the efficiency of the SupIM algorithm in support calculation, we propose the DFOM-CoNR and Pro-CoNR algorithms, which use the DFOM [54] and MatchingPro [7] algorithms to calculate the support, respectively.
3) CoNR-NoPBM: To validate the efficiency of the PBM structure, we propose CoNR-NoPBM, which does not use the PBM structure as well as the PSL generated based on the PBM structure.
4) CoNR-AET, CoNR-FET, and CoNR-BET: To verify the efficiency of FBEDFour in the candidate pattern generation, we design CoNR-AET, CoNR-FET and CoNR-BET,

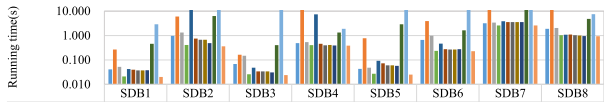
TABLE VII
PARAMETER SETTING

| Database | Name | Prefix pattern | $mincf$ |
|---|---|---|---|
| SDB1 | Babysale | $(ab)$ | 0.60 |
| SDB2 | E-shop | $(103)(10)$ | 0.60 |
| SDB3 | Movie | $(b)(c)$ | 0.80 |
| SDB4 | PM2.5 | $(10.0)$ | 0.60 |
| SDB5 | Game-sale | $(be)$ | 0.85 |
| SDB6 | Chicago-Crime | $(1)(3)$ | 0.30 |
| SDB7 | Credit | $(A)(A)$ | 0.60 |
| SDB8 | Letter | $(a)$ | 0.40 |

which use three enumeration tree strategies: AET [14], FET [14], and BET [14], respectively.

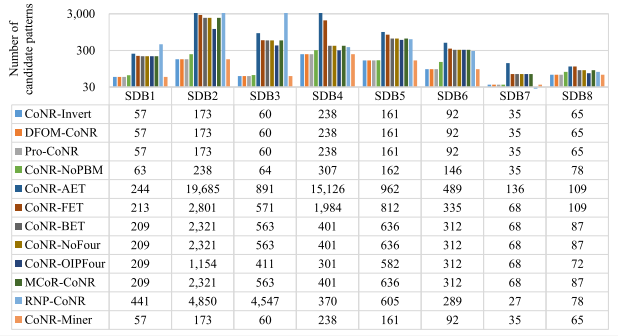
5) CoNR-NoFour and CoNR-OIPFour [44]: To validate the efficiency of the four pruning strategies in FBEDFour, we propose CoNR-NoFour and CoNR-OIPFour. CoNR-NoFour does not use the four pruning strategies of FBED-Four, while CoNR-OIPFour uses the four pruning strategies of OIP-Miner [44].
6) MCoR-CoNR [14] and RNP-CoNR [30]: To compare mining ability, we design the MCoR-CoNR and RNP-CoNR algorithms, which use the MCoR-Miner [14] and RNP-Miner [30] algorithms, respectively.
7) CoNR-w/o P1, CoNR-w/o P2, CoNR-w/o P3, and CoNR-w/o P4: To further validate the performance of the four pruning strategies, we propose CoNR-w/o P1, CoNR-w/o P2, CoNR-w/o P3, and CoNR-w/o P4, which do not employ prune strategies 1 to 4, respectively.
8) To validate the recommendation performance of CoNR-Miner, we select CoN-Miner as the competitive algorithm, which employs the same strategy as CoNR-Miner, but does not generate further co-occurrence rules; it only mines frequent patterns.

All experiments were performed on a computer equipped with an AMD Ryzen 7 4800H processor with Radeon Graphics, 16.0 GB RAM, and a Windows 10 64-bit operating system; the algorithms in the experiments can be run in the PyCharm environment.



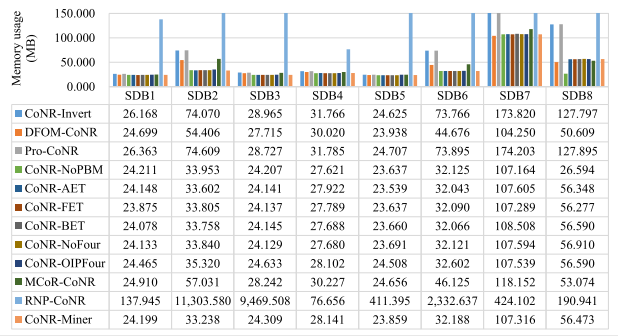

| | SDB1 | SDB2 | SDB3 | SDB4 | SDB5 | SDB6 | SDB7 | SDB8 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 0.041 | 0.971 | 0.068 | 0.490 | 0.043 | 0.662 | 3.163 | 1.868 |
| DFOM-CoNR | 0.265 | 6.001 | 0.163 | 16.283 | 0.775 | 3.914 | 180.053 | 14.196 |
| Pro-CoNR | 0.052 | 1.318 | 0.148 | 0.536 | 0.048 | 0.981 | 3.403 | 2.026 |
| CoNR-NoPBM | 0.021 | 0.408 | 0.026 | 0.402 | 0.027 | 0.236 | 2.591 | 1.033 |
| CoNR-AET | 0.042 | 13.076 | 0.048 | 7.356 | 0.092 | 0.466 | 3.815 | 1.082 |
| CoNR-FET | 0.040 | 0.750 | 0.034 | 0.460 | 0.073 | 0.275 | 3.544 | 1.100 |
| CoNR-BET | 0.037 | 0.670 | 0.034 | 0.395 | 0.060 | 0.266 | 3.531 | 1.028 |
| CoNR-NoFour | 0.037 | 0.675 | 0.034 | 0.411 | 0.060 | 0.268 | 3.522 | 1.000 |
| CoNR-OIPFour | 0.038 | 0.486 | 0.031 | 0.391 | 0.057 | 0.275 | 3.567 | 0.949 |
| MCoR-CoNR | 0.457 | 6.261 | 0.402 | 1.320 | 2.868 | 1.633 | 49.912 | 4.817 |
| RNP-CoNR | 2.903 | 13,853.897 | 7,194.960 | 1.887 | 105.457 | 156.034 | 17.202 | 7.473 |
| CoNR-Miner | 0.020 | 0.358 | 0.024 | 0.386 | 0.025 | 0.227 | 2.568 | 0.940 |

Fig. 4. Comparison of running time.




| | SDB1 | SDB2 | SDB3 | SDB4 | SDB5 | SDB6 | SDB7 | SDB8 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 57 | 173 | 60 | 238 | 161 | 92 | 35 | 65 |
| DFOM-CoNR | 57 | 173 | 60 | 238 | 161 | 92 | 35 | 65 |
| Pro-CoNR | 57 | 173 | 60 | 238 | 161 | 92 | 35 | 65 |
| CoNR-NoPBM | 63 | 238 | 64 | 307 | 162 | 146 | 35 | 78 |
| CoNR-AET | 244 | 19,685 | 891 | 15,126 | 962 | 489 | 136 | 109 |
| CoNR-FET | 213 | 2,801 | 571 | 1,984 | 812 | 335 | 68 | 109 |
| CoNR-BET | 209 | 2,321 | 563 | 401 | 636 | 312 | 68 | 87 |
| CoNR-NoFour | 209 | 2,321 | 563 | 401 | 636 | 312 | 68 | 87 |
| CoNR-OIPFour | 209 | 1,154 | 411 | 301 | 582 | 312 | 68 | 72 |
| MCoR-CoNR | 209 | 2,321 | 563 | 401 | 636 | 312 | 68 | 87 |
| RNP-CoNR | 441 | 4,850 | 4,547 | 370 | 605 | 289 | 27 | 78 |
| CoNR-Miner | 57 | 173 | 60 | 238 | 161 | 92 | 35 | 65 |

Fig. 5. Comparison of number of candidate patterns.




| | SDB1 | SDB2 | SDB3 | SDB4 | SDB5 | SDB6 | SDB7 | SDB8 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 26.168 | 74.070 | 28.965 | 31.766 | 24.625 | 73.766 | 173.820 | 127.797 |
| DFOM-CoNR | 24.699 | 54.406 | 27.715 | 30.020 | 23.938 | 44.676 | 104.250 | 50.609 |
| Pro-CoNR | 26.363 | 74.609 | 28.727 | 31.785 | 24.707 | 73.895 | 174.203 | 127.895 |
| CoNR-NoPBM | 24.211 | 33.953 | 24.207 | 27.621 | 23.637 | 32.125 | 107.164 | 26.594 |
| CoNR-AET | 24.148 | 33.602 | 24.141 | 27.922 | 23.539 | 32.043 | 107.605 | 56.348 |
| CoNR-FET | 23.875 | 33.805 | 24.137 | 27.789 | 23.637 | 32.090 | 107.289 | 56.277 |
| CoNR-BET | 24.078 | 33.758 | 24.145 | 27.688 | 23.660 | 32.066 | 108.508 | 56.590 |
| CoNR-NoFour | 24.133 | 33.840 | 24.129 | 27.680 | 23.691 | 32.121 | 107.594 | 56.910 |
| CoNR-OIPFour | 24.465 | 35.320 | 24.633 | 28.102 | 24.508 | 32.602 | 107.539 | 56.590 |
| MCoR-CoNR | 24.910 | 57.031 | 28.242 | 30.227 | 24.656 | 46.125 | 118.152 | 53.074 |
| RNP-CoNR | 137.945 | 11,303.580 | 9,469.508 | 76.656 | 411.395 | 2,332.637 | 424.102 | 190.941 |
| CoNR-Miner | 24.199 | 33.238 | 24.309 | 28.141 | 23.859 | 32.188 | 107.316 | 56.473 |

Fig. 6. Comparison of memory usage.

## B. Mining Performance and Analysis

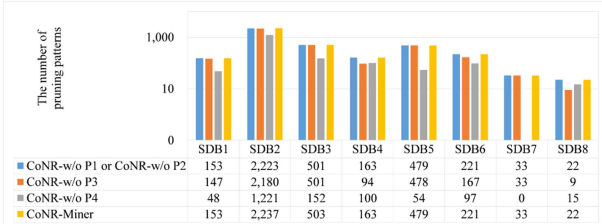
To validate the performance of CoNR-Miner, we use eight databases and CoNR-Invert, DFOM-CoNR, Pro-CoNR, CoNR-NoPBM, CoNR-AET, CoNR-FET, CoNR-BET, CoNR-NoFour, CoNR-OIPFour, MCoR-CoNR, and RNP-CoNR, as competitive algorithms. Since the number of items and the size of the itemsets vary in each database, the parameter settings of the databases are as shown in Table VII, to make the algorithm's performance on each database easy to observe and analyze. The number of CoNRs mined for the eight databases are 28, 66, 39, 21, 73, 13, 16, and 9, respectively. The comparisons of running time, number of candidate patterns, and memory usage are shown in Figs. 4 to 6, respectively.

The following observations can be made.

1) CoNR-Miner outperforms CoNR-Invert, which indicates the advantage of integer mask dictionary. For example, for SDB1 in Fig. 4, CoNR-Miner takes 0.020 s, while CoNR-Invert takes 0.041 s. The reason for this is that CoNR-Invert uses position index dictionary, which takes longer to match than integer mask dictionary; the integer mask store is binary and computer ultimately uses the binary format for matching. In addition, Fig. 6 shows that CoNR-Miner consumes less memory than CoNR-Invert. For example, in SDB1, CoNR-Miner consumes 24.199 MB, while CoNR-Invert consumes 26.168 MB. The reason for this is that the integer mask can efficiently record an item's occurrence position in a sequence, which consumes less memory than the position index. Hence, CoNR-Miner runs faster than CoNR-Invert.
2) CoNR-Miner performs better than DFOM-CoNR and Pro-CoNR, which indicates the efficiency of the SupIM algorithm. For example, for SDB6 in Fig. 4, CoNR-Miner takes 0.227 s, while DFOM-CoNR and Pro-CoNR take

3.914 s and 0.981 s, respectively. CoNR-Miner is 17.2 times and 4.3 times faster than DFOM-CoNR and Pro-CoNR, respectively. The reason for this is that CoNR-Miner uses SupIM to calculate the superpattern support based on the integer mask dictionary of the subpatterns; it does not need to scan the database repeatedly. Therefore, SupIM is more efficient than DFOM and MatchingPro. Meanwhile, Fig. 6 shows that CoNR-Miner consumes less memory than DFOM-CoNR and Pro-CoNR. The reason for this is that CoNR-Miner uses integer mask dictionary, thereby reducing the memory usage. These two algorithms use the same candidate pattern generation method but different support calculation methods, proving that SupIM can effectively improve the mining performance. Hence, CoNR-Miner achieves a better performance.

3) CoNR-Miner is superior to CoNR-NoPBM, which indicates that the PBM structure can effectively improve the algorithm performance. For example, for SDB4 in Figs. 4 and 5, CoNR-NoPBM takes 0.402 s and generates 307 candidate patterns. CoNR-Miner takes 0.386 s and generates 238 candidate patterns, which reduces the number of support calculations by 69 times, compared with CoNR-NoPBM. However, Fig. 6 shows that CoNR-Miner consumes more memory than CoNR-NoPBM. For example, for SDB4, CoNR-Miner consumes 28.141 MB and CoNR-NoPBM consumes 27.621 MB. The reason for this is that the PBM structure leads to increased memory usage. Hence, CoNR-Miner outperforms CoNR-NoPBM.
4) CoNR-Miner outperforms CoNR-AET, CoNR-FET, and CoNR-BET, which indicates that FBEDFour can effectively reduce the number of candidate patterns. Figs. 4 and 5 show that CoNR-Miner runs faster and generates fewer candidate patterns than CoNR-AET, CoNR-FET, and CoNR-BET. For example, for SDB5, CoNR-AET, CoNR-FET, and CoNR-BET take 0.092 s, 0.073 s, and 0.060 s, and generate 962, 812, and 636 candidate patterns, respectively. However, CoNR-Miner takes 0.025 s and generates 161 candidate patterns. Hence, CoNR-Miner runs faster than CoNR-AET, CoNR-FET, and CoNR-BET.
5) CoNR-Miner outperforms CoNR-NoFour and CoNR-OIPFour, which indicates that the four pruning strategies in FBEDFour can effectively reduce the number of candidate patterns. For example, for SDB2 in Fig. 4, CoNR-NoFour and CoNR-OIPFour run in 0.675 s and 0.486 s, and generate 2,321 and 1,154 candidate patterns, respectively. However, CoNR-Miner runs in 0.358 s and generates 173 candidate patterns. Hence, CoNR-Miner outperforms CoNR-NoFour and CoNR-OIPFour.
6) CoNR-Miner runs faster, generates fewer candidate patterns, and consumes less memory than MCoR-CoNR and RNP-CoNR. For example, for SDB3, MCoR-CoNR takes 0.402 s, generates 563 candidate patterns, and consumes 28.242 MB, RNP-CoNR takes 7,194.960 s, generates 4,547 candidate patterns, and consumes 9,469.508 MB, while CoNR-Miner takes 0.024 s, generates 60 candidate patterns, and consumes 24.309 MB. The reasons for this are as follows. At data preprocessing stage, MCoR-CoNR and RNP-CoNR both employ position index dictionary, while CoNR-Miner uses integer mask dictionary. At support calculation stage, MCoR-CoNR employs DBI for layer-by-layer scanning of the position index dictionary for pattern matching. RNP-CoNR adopts CSC to calculate pattern support using the position index dictionary of subpatterns. CoNR-Miner employs SupIM to directly calculate pattern support using the integer mask dictionary of subpatterns. As described in the analysis in Section V-B, integer mask dictionary outperforms position index dictionary. Hence, CoNR-Miner achieves the best performance.




| | SDB1 | SDB2 | SDB3 | SDB4 | SDB5 | SDB6 | SDB7 | SDB8 |
|---|---|---|---|---|---|---|---|---|
| CoNR-w/o P1 or CoNR-w/o P2 | 153 | 2,223 | 501 | 163 | 479 | 221 | 33 | 22 |
| CoNR-w/o P3 | 147 | 2,180 | 501 | 94 | 478 | 167 | 33 | 9 |
| CoNR-w/o P4 | 48 | 1,221 | 152 | 100 | 54 | 97 | 0 | 15 |
| CoNR-Miner | 153 | 2,237 | 503 | 163 | 479 | 221 | 33 | 22 |

Fig. 7. Comparison of pruning performance.

## C. Ablation

To further analyze the performance of four pruning strategies, we select CoNR-w/o P1 to CoNR-w/o P4 as competitive algorithms. The results are presented in Fig. 7.

For SDB2, CoNR-w/o P1 or CoNR-w/o P2 prunes 2,223 patterns, CoNR-w/o P3 prunes 2,180 patterns, and CoNR-w/o P4 prunes 1,221 patterns; all are fewer than patterns pruned by CoNR-Miner. Moreover, CoNR-Miner prunes 14, 57, and 1,016 more patterns than CoNR-w/o P1 or P2, CoNR-w/o P3, and CoNR-w/o P4, respectively. Pruning strategies 1, 2, and 3 serves as prepruning steps for pruning strategy 4, avoiding multiple superpattern pruning strategy evaluations, thereby reducing running time. Hence, the four pruning strategies are effective.

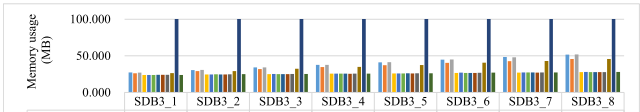
## D. Scalability

To analyze the scalability of CoNR-Miner, CoNR-Invert, DFOM-CoNR, Pro-CoNR, CoNR-NoPBM, CoNR-AET, CoNR-FET, CoNR-BET, CoNR-NoFour, CoNR-OIPFour, MCoR-CoNR, and RNP-CoNR are selected. We select database SDB3 in Table VI and create SDB3_1, SDB3_2, SDB3_3, SDB3_4, SDB3_5, SDB3_6, SDB3_7, and SDB3_8, with sizes 1-8 times that of SDB3. We set $minc f$=0.9 and the prefix pattern as $(b)(c)$, and seven CoNRs are mined in each of them. Comparisons of running time and memory usage are shown in Figs. 8 and 9, respectively.

The following observations can be made. CoNR-Miner has better scalability than other competitive algorithms. The increase in running time of CoNR-Miner is lower than competitive other algorithms when the size of the database increases from SDB3_1 to SDB3_8. From Fig. 8, the increase in running time for CoNR-Invert, DFOM-CoNR, Pro-CoNR, CoNR-NoPBM, CoNR-AET, CoNR-FET, CoNR-BET, CoNR-NoFour, CoNR-OIPFour, MCoR-CoNR, and RNP-CoNR is


| | SDB3_1 | SDB3_2 | SDB3_3 | SDB3_4 | SDB3_5 | SDB3_6 | SDB3_7 | SDB3_8 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 0.044 | 0.092 | 0.130 | 0.171 | 0.220 | 0.256 | 0.313 | 0.354 |
| DFOM-CoNR | 0.115 | 0.219 | 0.333 | 0.445 | 0.546 | 0.658 | 0.768 | 0.893 |
| Pro-CoNR | 0.088 | 0.174 | 0.254 | 0.334 | 0.428 | 0.517 | 0.603 | 0.698 |
| CoNR-NoPBM | 0.014 | 0.024 | 0.033 | 0.043 | 0.052 | 0.062 | 0.071 | 0.079 |
| CoNR-AET | 0.023 | 0.043 | 0.060 | 0.077 | 0.098 | 0.115 | 0.137 | 0.153 |
| CoNR-FET | 0.017 | 0.027 | 0.039 | 0.052 | 0.061 | 0.074 | 0.087 | 0.100 |
| CoNR-BET | 0.016 | 0.027 | 0.038 | 0.051 | 0.061 | 0.074 | 0.085 | 0.097 |
| CoNR-NoFour | 0.017 | 0.027 | 0.040 | 0.052 | 0.063 | 0.075 | 0.086 | 0.098 |
| CoNR-OIPFour | 0.014 | 0.025 | 0.033 | 0.044 | 0.053 | 0.064 | 0.073 | 0.084 |
| MCoR-CoNR | 0.124 | 0.255 | 0.375 | 0.506 | 0.642 | 0.760 | 0.888 | 1.027 |
| RNP-CoNR | 115.367 | 206.169 | 297.852 | 385.217 | 473.914 | 564.255 | 652.151 | 740.286 |
| CoNR-Miner | 0.013 | 0.021 | 0.030 | 0.039 | 0.047 | 0.057 | 0.065 | 0.073 |

Fig. 8. Comparison of running time for different database sizes.




| | SDB3_1 | SDB3_2 | SDB3_3 | SDB3_4 | SDB3_5 | SDB3_6 | SDB3_7 | SDB3_8 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 27.215 | 30.484 | 34.207 | 37.742 | 41.063 | 44.801 | 48.430 | 51.730 |
| DFOM-CoNR | 26.121 | 29.109 | 31.941 | 34.629 | 37.129 | 40.359 | 42.660 | 45.590 |
| Pro-CoNR | 26.977 | 30.793 | 34.266 | 37.668 | 41.230 | 44.926 | 48.012 | 51.793 |
| CoNR-NoPBM | 23.809 | 24.676 | 25.117 | 25.648 | 25.957 | 26.691 | 27.086 | 27.672 |
| CoNR-AET | 23.879 | 24.527 | 25.266 | 25.688 | 26.027 | 27.051 | 27.379 | 27.930 |
| CoNR-FET | 23.902 | 24.684 | 25.090 | 25.711 | 25.938 | 26.734 | 27.305 | 27.668 |
| CoNR-BET | 24.105 | 24.664 | 25.109 | 25.648 | 26.145 | 26.641 | 27.223 | 27.840 |
| CoNR-NoFour | 24.004 | 24.602 | 24.984 | 25.473 | 25.879 | 26.625 | 27.172 | 27.785 |
| CoNR-OIPFour | 24.043 | 24.691 | 25.289 | 25.746 | 26.117 | 26.820 | 27.383 | 27.789 |
| MCoR-CoNR | 26.395 | 29.168 | 32.383 | 34.934 | 37.504 | 40.625 | 42.926 | 45.801 |
| RNP-CoNR | 459.301 | 841.246 | 1,225.852 | 1,599.629 | 1,964.410 | 2,359.223 | 2,726.711 | 3,101.293 |
| CoNR-Miner | 23.981 | 24.906 | 25.164 | 25.809 | 26.191 | 27.047 | 27.270 | 27.895 |

Fig. 9. Comparison of memory usage for different database sizes.

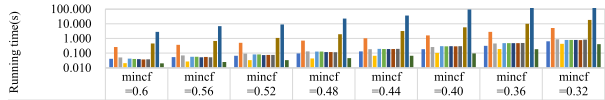
0.354/0.044=8.045, 0.893/0.115=7.765, 0.698/0.088=7.932, 0.079/0.014=5.643, 0.153/0.023=6.652, 0.100/0.017=5.882, 0.097/0.016=6.063, 0.098/0.017=5.765, 0.084/0.014=6.000, 1.027/0.124=8.282, and 740.286/115.367=6.417, respectively. The increase of running time of CoNR-Miner is 0.073/0.013=5.615, which is smaller than that of the above algorithms. Moreover, from Fig. 9, the memory usage of CoNR-Miner increases linearly with the growth of the database. For example, the memory usage is 23.981 MB for SDB3_1 and 27.895 MB for SDB3_8, i.e., growing by a factor of 27.895/23.981=1.163. Thus, the growth rate of memory usage is lower than that of the database size. Hence, CoNR-Miner has strong scalability.

### *E. Sensitivity Analysis*

1) Sensitivity analysis of $mincf$

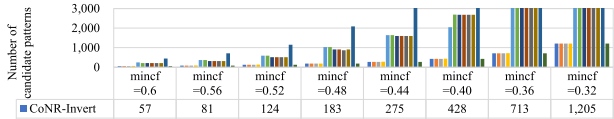
To assess the influence of different $mincf$, we select SDB1 as the experimental database and choose CoNR-Invert, DFOM-CoNR, Pro-CoNR, CoNR-NoPBM, CoNR-AET, CoNR-FET, CoNR-BET, CoNR-NoFour, CoNR-OIPFour, MCoR-CoNR, and RNP-CoNR as competitive algorithms. The prefix pattern is set to $(ab)$, and $mincf$ = 0.6, 0.56, 0.52, 0.48, 0.44, 0.40, 0.36, and 0.32, and the numbers of CoNRs mined are 28, 42, 69, 114, 179, 286, 507, and 876, respectively. Comparisons of running time, numbers of candidate patterns, and memory usage are shown in Figs. 10 to 12, respectively.

The conclusions are as follows. As $mincf$ decreases, the numbers of CoNRs and candidate patterns, running time, and memory usage all show an increasing trend. For example, when



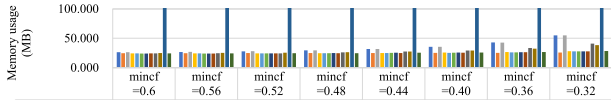

| | mincf =0.6 | mincf =0.56 | mincf =0.52 | mincf =0.48 | mincf =0.44 | mincf =0.40 | mincf =0.36 | mincf =0.32 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 0.041 | 0.053 | 0.067 | 0.095 | 0.131 | 0.187 | 0.317 | 0.647 |
| DFOM-CoNR | 0.262 | 0.373 | 0.512 | 0.716 | 1.033 | 1.637 | 2.812 | 5.107 |
| Pro-CoNR | 0.051 | 0.069 | 0.093 | 0.132 | 0.184 | 0.264 | 0.455 | 0.886 |
| CoNR-NoPBM | 0.021 | 0.027 | 0.033 | 0.044 | 0.066 | 0.103 | 0.191 | 0.428 |
| CoNR-AET | 0.042 | 0.057 | 0.082 | 0.128 | 0.194 | 0.296 | 0.485 | 0.791 |
| CoNR-FET | 0.040 | 0.057 | 0.082 | 0.128 | 0.192 | 0.294 | 0.479 | 0.787 |
| CoNR-BET | 0.039 | 0.052 | 0.074 | 0.118 | 0.192 | 0.295 | 0.480 | 0.785 |
| CoNR-NoFour | 0.037 | 0.055 | 0.076 | 0.119 | 0.189 | 0.292 | 0.475 | 0.781 |
| CoNR-OIPFour | 0.038 | 0.052 | 0.076 | 0.115 | 0.195 | 0.302 | 0.495 | 0.837 |
| MCoR-CoNR | 0.457 | 0.671 | 1.087 | 1.949 | 3.233 | 5.663 | 9.958 | 18.131 |
| RNP-CoNR | 2.818 | 6.819 | 8.923 | 22.593 | 36.017 | 94.184 | 188.155 | 532.054 |
| CoNR-Miner | 0.020 | 0.025 | 0.033 | 0.046 | 0.067 | 0.097 | 0.186 | 0.413 |

Fig. 10. Comparison of running time with different $mincf$.




| | mincf =0.6 | mincf =0.56 | mincf =0.52 | mincf =0.48 | mincf =0.44 | mincf =0.40 | mincf =0.36 | mincf =0.32 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 57 | 81 | 124 | 183 | 275 | 428 | 713 | 1,205 |
| DFOM-CoNR | 57 | 81 | 124 | 183 | 275 | 428 | 713 | 1,205 |
| Pro-CoNR | 57 | 81 | 124 | 183 | 275 | 428 | 713 | 1,205 |
| CoNR-NoPBM | 63 | 89 | 132 | 184 | 280 | 435 | 720 | 1,213 |
| CoNR-AET | 244 | 366 | 590 | 1,027 | 1,639 | 2,047 | 4,704 | 8,133 |
| CoNR-FET | 213 | 366 | 590 | 1,027 | 1,639 | 2,688 | 4,704 | 8,133 |
| CoNR-BET | 209 | 316 | 513 | 907 | 1,594 | 2,675 | 4,678 | 8,088 |
| CoNR-NoFour | 209 | 316 | 513 | 907 | 1,594 | 2,675 | 4,678 | 8,088 |
| CoNR-OIPFour | 209 | 316 | 513 | 864 | 1,594 | 2,675 | 4,678 | 8,088 |
| MCoR-CoNR | 209 | 316 | 513 | 907 | 1,594 | 2,675 | 4,678 | 8,088 |
| RNP-CoNR | 441 | 712 | 1,150 | 2,090 | 3,506 | 5,911 | 10,400 | 18,457 |
| CoNR-Miner | 57 | 81 | 124 | 183 | 275 | 428 | 713 | 1,205 |

Fig. 11. Comparison of numbers of candidate patterns with different $mincf$.




| | mincf =0.6 | mincf =0.56 | mincf =0.52 | mincf =0.48 | mincf =0.44 | mincf =0.40 | mincf =0.36 | mincf =0.32 |
|---|---|---|---|---|---|---|---|---|
| CoNR-Invert | 26.168 | 26.684 | 27.719 | 29.383 | 31.723 | 35.574 | 42.816 | 54.840 |
| DFOM-CoNR | 24.563 | 24.656 | 24.711 | 24.840 | 24.758 | 25.020 | 25.258 | 25.563 |
| Pro-CoNR | 26.387 | 26.961 | 28.047 | 29.563 | 31.844 | 35.535 | 42.645 | 54.863 |
| CoNR-NoPBM | 24.211 | 24.379 | 24.406 | 24.684 | 24.750 | 25.816 | 26.461 | 28.059 |
| CoNR-AET | 24.148 | 24.309 | 24.379 | 24.668 | 24.965 | 24.984 | 25.973 | 27.648 |
| CoNR-FET | 23.875 | 24.109 | 24.238 | 24.551 | 24.984 | 25.320 | 25.988 | 27.641 |
| CoNR-BET | 24.098 | 24.020 | 24.250 | 24.902 | 25.270 | 25.574 | 26.137 | 27.609 |
| CoNR-NoFour | 24.133 | 24.098 | 24.314 | 24.609 | 24.922 | 25.535 | 26.324 | 27.641 |
| CoNR-OIPFour | 24.379 | 24.605 | 24.125 | 25.914 | 27.441 | 29.207 | 33.434 | 40.746 |
| MCoR-CoNR | 24.910 | 25.125 | 25.406 | 26.219 | 27.453 | 29.184 | 32.336 | 38.199 |
| RNP-CoNR | 137.707 | 231.066 | 280.156 | 480.250 | 638.152 | 1,082.895 | 1,577.738 | 2,715.824 |
| CoNR-Miner | 24.199 | 24.305 | 24.449 | 24.664 | 25.359 | 25.746 | 26.637 | 28.266 |

Fig. 12. Comparison of memory usage with different $mincf$.

$mincf$ =0.6, CoNR-Miner mines 28 CoNRs, generates 57 candidate patterns, takes 0.020s, and consumes 24.199MB. When $mincf$ =0.32, CoNR-Miner mines 876 CoNRs, generates 1,205 candidate patterns, takes 0.413s, and consumes 28.266MB. This phenomenon can be found in other competitive algorithms. The reason is as follows. As $mincf$ decreases, $minsup$ decreases according to the minimum support strategy. More CoNRs are mined, more candidate patterns are generated, and running time and memory usage increase. More importantly, CoNR-Miner outperforms other competitive algorithms for all of the tested $mincf$ values.

2) Sensitivity analysis of prefix length

To validate the impact of prefix length, SDB1 from Table VI is selected as the experimental database. The $mincf$ value is set to 0.3. Table VIII shows the comparison of mining results for different prefix lengths.

The following conclusions can be drawn. As the prefix size and length increase, the number of candidate patterns, running

TABLE VIII
COMPARISON OF MINING RESULTS FOR DIFFERENT PREFIX LENGTHS

| Prefix pattern | Support | Number of candidate patterns | Running time (s) | Number of CoNRs |
|---|---|---|---|---|
| $(b)$ | 967 | 2,798 | 1.438 | 2,174 |
| $(b)(b)$ | 622 | 678 | 0.152 | 463 |
| $(b)(b)(a)$ | 544 | 557 | 0.109 | 369 |
| $(b)(b)(a)(a)$ | 460 | 445 | 0.080 | 279 |
| $(b)(b)(a)(a)(a)$ | 385 | 357 | 0.067 | 216 |
| $(b)(b)(a)(a)(a)(a)$ | 307 | 331 | 0.058 | 195 |
| $(b)(b)(a)(a)(a)(a)(c)$ | 217 | 161 | 0.035 | 88 |
| $(b)(b)(a)(a)(a)(a)(c)(c)$ | 148 | 137 | 0.034 | 74 |

TABLE IX
COMPARISON OF MINING RESULTS FOR DIFFERENT $Avg(i)$

| Database | $Avg(i)$ | Number of candidate patterns | Number of pruning patterns | Running time (s) | Number of CoNRs |
|---|---|---|---|---|---|
| SDB1L1 | 1.9425 | 28 | 28 | 0.003 | 20 |
| SDB1L2 | 2.2017 | 132 | 191 | 0.008 | 103 |
| SDB1L3 | 2.3321 | 160 | 351 | 0.014 | 122 |
| SDB1L4 | 2.4731 | 271 | 570 | 0.019 | 215 |
| SDB1L5 | 2.6138 | 284 | 678 | 0.020 | 218 |
| SDB1L6 | 2.7819 | 282 | 649 | 0.017 | 203 |
| SDB1L7 | 2.9656 | 334 | 923 | 0.022 | 247 |
| SDB1L8 | 3.3731 | 808 | 3,778 | 0.083 | 632 |

time and number of CoNRs all show a downward trend. For example, when the prefix size and length are one, its support is 967 and CoNR-Miner generates 2,798 candidate patterns, takes 1.438 s, and mines 2,174 CoNRs. When the prefix size and length are eight, its support is 148 and CoNR-Miner generates 137 candidate patterns, takes 0.034 s, and mines 74 CoNRs. This phenomenon can be explained by the principle of anti-monotonicity: the support of any superpattern (i.e. a pattern containing the prefix) is not greater than the support of the prefix pattern; therefore, as the prefix length increase, the prefix pattern support gradually decreases, and the $minsup$ based on the minimum support strategy decreases accordingly. However, although the $minsup$ for long prefix patterns is smaller than that for short prefix patterns, the anti-monotonicity property implies that longer patterns are inherently sparse within the data. Consequently, the upper bound of support for any candidate superpattern is strictly confined to a narrow range, and the set of candidate patterns that contain the given prefix correspondingly shrinks. Thus, the number of candidate patterns satisfying $minsup$ decreases significantly. Hence, the number of candidate patterns, running time, and number of CoNRs all show a downward trend.

3) Sensitivity analysis of $Avg(i)$

To report the influence of $Avg(i)$, SDB1 from Table VI is selected as the experimental database. To facilitate the comparison, SDB1 is randomly divided into eight equal subsets, i.e. SDB1L1, SDB1L2, SDB1L3, SDB1L4, SDB1L5, SDB1L6, SDB1L7, and SDB1L8. The prefix pattern and $mincf$ are set to $(a)$ and 0.3, respectively. The comparison of mining results for different $Avg(i)$ is shown in Table IX.

The following observations are made. As $Avg(i)$ increases, the numbers of candidate patterns, pruning patterns, and CoNRs, and running time generally show an upward trend. For example, when $Avg(i)$ of SDB1L1 is 1.9425, CoNR-Miner generates 28 candidate patterns, prunes 28 patterns, takes 0.003s, and mines 20 CoNRs; when $Avg(i)$ of SDB1L8 is 3.3731, CoNR-Miner generates 808 candidate patterns, prunes 3,778 patterns, takes 0.083s, and mines 632 CoNRs. The reason is as follows. As $Avg(i)$ increases, the itemset structure becomes more complex and more redundant patterns are generated. Hence, the numbers of candidate patterns, pruning patterns and CoNRs, and running time increase significantly.

TABLE X
PARAMETERS OF RECOMMENDATION PERFORMANCE

| Database | Name | Prefix pattern | $mincf$ | $minsup$ |
|---|---|---|---|---|
| SDB1 | Babysale | $(ab)$ | 0.60 | 520 |
| SDB2 | E-shop | $(103)(132)$ | 0.60 | 450 |
| SDB3 | Movie | $(de)$ | 0.62 | 310 |
| SDB4 | PM 2.5 | $(21.0)$ | 0.60 | 180 |
| SDB5 | Game-sale | $(de)$ | 0.85 | 600 |
| SDB6 | Chicago-Crime7 | $(18)$ | 0.40 | 900 |
| SDB7 | Credit | $(A)(A)$ | 0.60 | 100,000 |
| SDB8 | Letter | $(a)$ | 0.40 | 5,650 |

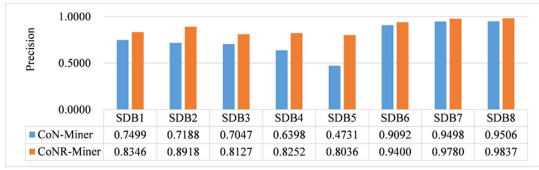



Fig. 13. Comparison of precision.

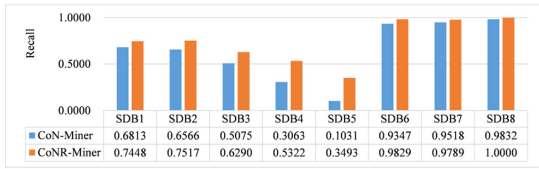



Fig. 14. Comparison of recall.

## F. Case Study

To validate the recommendation performance of CoNR-Miner, we select CoN-Miner as the competitive algorithm. We select eight databases from Table VI as experimental databases, each database being divided into training and testing sets at an 8:2 ratio. The settings for the prefix patterns $mincf$ and $minsup$ are shown in Table X.

There are three commonly used evaluation metrics for recommendation performance experiments: precision $Pr = \frac{TP}{TP+FP}$, recall $Re = \frac{TP}{TP+FN}$, and F1-score $F1 = \frac{2\times Pr\times Re}{Pr+Re}$, where $TP$, $FP$, and $FN$ are the numbers of correct recommendations, incorrect recommendations, and relevant items that are not included in the recommendation list, respectively.

Considering the limited number of sequences in each database, we use 5-fold cross-validation for each database, to ensure more comprehensive and reliable evaluation results. Comparisons of precision, recall, and F1-score are shown in Figs. 13 to 15, respectively.

The results give rise to the following observations. CoNR-Miner performs better than CoN-Miner in terms of recommendation performance. From Figs. 13 to 15, the precision, recall,

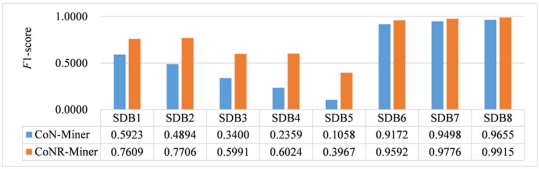



Fig. 15. Comparison of F1-score.

and F1-score of CoN-Miner in SDB4 are 0.6398, 0.3063, and 0.2359, respectively, while those of CoNR-Miner are 0.8252, 0.5322, and 0.6024, respectively, i.e., all of which are higher than those of CoN-Miner. This phenomenon can also be observed in the other seven databases. CoNR-Miner can effectively discover strong co-occurrence rules with higher confidence, i.e., behavior patterns with common interests and preferences of users. Therefore, CoNR-Miner has better recommendation performance than CoN-Miner.

## VI. Conclusion

This paper explores self-adaptive co-cccurrence nonoverlapping sequential rule mining and proposes the CoNR-Miner algorithm. CoNR-Miner comprises three parts: data preprocessing, support calculation, and candidate pattern generation. In data preprocessing, we propose the integer mask dictionary to convert the original sequential database into an integer mask. In support calculation, we employ several bitwise operations on the integer masks of subpatterns and items to improve efficiency. In candidate pattern generation, we propose the FBED strategy to generate candidate patterns and four pruning strategies to further reduce candidate patterns. To evaluate the efficiency of CoNR-Miner, we conduct experiments on eight databases and 16 competitive algorithms. The experimental results demonstrate that CoNR-Miner outperforms all other competitive algorithms. More importantly, it yields superior recommendation performance compared to methods that only mine frequent co-occurrence patterns.

In the future, the following work can be considered. 1) This paper proposes a CoNR mining method based on a given prefix pattern in sequential databases. However, our work has not fully considered dynamic data. Future research can be extended to dynamic databases and explore the automatic generation of prefixes. 2) CoNR mining does not incorporate user interests. In practical applications, mining high-value patterns that align with user interests is of greater significance. Thus, future research can integrate user interest into CoNR mining. 3) This paper primarily focuses on mining all CoNRs. Nevertheless, in the context of massive data, the efficient extraction of top-*k* representative rules is more critical than mining all rules. Future work may investigate efficient top-*k* CoNR mining methods.

## References

[1] Z. Chen, W. Gan, G. Huang, Z. Qi, Y. Li, and P. S. Yu, "TALENT: Targeted mining of non-overlapping sequential patterns," *ACM Trans. Manage. Inf. Syst.*, vol. 16, no. 4, pp. 1–34, 2025.

[2] F. Min, Z.-H. Zhang, W.-J. Zhai, and R.-P. Shen, "Frequent pattern discovery with tri-partition alphabets," *Inf. Sci.*, vol. 507, pp. 715–732, 2020.

[3] Y. Wu et al., "NTP-Miner: Nonoverlapping three-way sequential pattern mining," *ACM Trans. Knowl. Discov. Data*, vol. 16, no. 3, pp. 1–21, 2022.

[4] Y. Li et al., "OSP-Miner: Mining one-off weak-gap strong sequential patterns," *Inf. Sci.*, vol. 730, 2026, Art. no. 122871.

[5] C. Ma, H. Q. Vu, J. Wang, V.-H. Trieu, and G. Li, "Understanding residents' behavior for smart city management by sequential and periodic pattern mining," *IEEE Trans. Comput. Social Syst.*, vol. 11, no. 1, pp. 1260–1276, Feb. 2024.

[6] Z. He, S. Zhang, F. Gu, and J. Wu, "Mining conditional discriminative sequential patterns," *Inf. Sci.*, vol. 478, pp. 524–539, 2019.

[7] Y. Wu, Y. Wang, Y. Li, X. Zhu, and X. Wu, "Top-k self-adaptive contrast sequential pattern mining," *IEEE Trans. Cybern.*, vol. 52, no. 11, pp. 11819–11833, Nov. 2022.

[8] X. Gao, Y. Gong, T. Xu, J. Lü, Y. Zhao, and X. Dong, "Toward better structure and constraint to mine negative sequential patterns," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 2, pp. 571–585, Feb. 2023.

[9] Y. Li et al., "Mining repetitive negative sequential patterns with gap constraints," *ACM Trans. Knowl. Discov. Data*, vol. 19, no. 4, pp. 86:1–86:29, 2025.

[10] W. Gan, G. Huang, J. Weng, T. Gu, and P. S. Yu, "Towards target sequential rules," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 6, pp. 3766–3780, Jun. 2025.

[11] K. Hu, W. Gan, S. Huang, H. Peng, and P. Fournier-Viger, "Targeted mining of contiguous sequential patterns," *Inf. Sci.*, vol. 653, 2024, Art. no. 119791.

[12] L. Li, W. Zuba, G. Loukides, S. P. Pissis, and M. Matsangidou, "Scalable order-preserving pattern mining," in *Proc. IEEE Int. Conf. Data Mining*, 2024, pp. 211–220.

[13] Y. Li et al., "OPF-Miner: Order-preserving pattern mining with forgetting mechanism for time series," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 8981–8995, Dec. 2024.

[14] Y. Li et al., "MCoR-Miner: Maximal co-occurrence nonoverlapping sequential rule mining," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9531–9546, Sep. 2023.

[15] L. Wang, V. Tran, and T. Do, "A clique-querying mining framework for discovering high utility co-location patterns without generating candidates," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 1, pp. 1–42, 2023.

[16] J. Li, L. Wang, P. Yang, and L. Zhou, "A novel algorithm for efficiently mining spatial multi-level co-location patterns," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4361–4374, Sep. 2024.

[17] C. H. Mooney and J. F. Roddick, "Sequential pattern mining–approaches and algorithms," *ACM Comput. Surv.*, vol. 45, no. 2, pp. 19:1–19:39, 2013.

[18] W. Gan, J. C.-W. Lin, P. Fournier-Viger, H.-C. Chao, and P. S. Yu, "A survey of parallel sequential pattern mining," *ACM Trans. Knowl. Discov. Data*, vol. 13, no. 3, pp. 25:1–25:34, 2019.

[19] Y. Wu, Y. Tong, X. Zhu, and X. Wu, "NOSEP: Nonoverlapping sequence pattern mining with gap constraints," *IEEE Trans. Cybern.*, vol. 48, no. 10, pp. 2809–2822, Oct. 2018.

[20] C. Li, Q. Yang, J. Wang, and M. Li, "Efficient mining of gap-constrained subsequences and its various applications," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, Mar. 2012.

[21] X. Wu, X. Zhu, Y. He, and A. N. Arslan, "PMBC: Pattern mining from biological sequences with wildcard constraints," *Comput. Biol. Med.*, vol. 43, no. 5, pp. 481–492, 2013.

[22] H. T. Lam, F. Moerchen, D. Fradkin, and T. Calders, "Mining compressing sequential patterns," *Statist. Anal. Data Mining*, vol. 7, no. 1, pp. 34–52, 2014.

[23] C. Sun et al., "SN-RNSP: Mining self-adaptive nonoverlapping repetitive negative sequential patterns in transaction sequences," *Knowl. Based Syst.*, vol. 287, 2024, Art. no. 111449.

[24] X. Dong, Y. Gong, and L. Cao, "e-RNSP: An efficient method for mining repetition negative sequential patterns," *IEEE Trans. Cybern.*, vol. 50, no. 5, pp. 2084–2096, May 2020.

[25] C. Zhang, X. Meng, and H. Hao, "MNSPM: Merging-based nonoverlapping gap-constrained sequential pattern mining," *Inf. Process. Manag.*, vol. 63, no. 5, 2026, Art. no. 104682.

[26] P. Fournier-Viger, C.-W. Wu, V. S. Tseng, L. Cao, and R. Nkambou, "Mining partially-ordered sequential rules common to multiple sequences," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 8, pp. 2203–2216, Aug. 2015.

[27] Y. Li, X. Gao, J. Li, R. Gao, P. Fournier-Viger, and Y. Wu, "CoTP-Miner: Co-occurrence three-way sequential pattern mining," *Knowl. Based Syst.*, vol. 332, 2025, Art. no. 114803.

[28] K. Nam, "Conversion paths of online consumers: A sequential pattern mining approach," *Expert Syst. Appl.*, vol. 202, 2022, Art. no. 117253.

[29] H. W. Liu, J. Z. Wu, and Y. H. Wang, "Uncovering insights for new car recommendations with sequence pattern mining on mobile applications," *Appl. Sci.*, vol. 13, no. 11, 2023, Art. no. 6386.
[30] M. Geng et al., "RNP-Miner: Repetitive nonoverlapping sequential pattern mining," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4874–4889, Sep. 2024.
[31] P. Sra and S. Chand, "A reinduction-based approach for efficient high utility itemset mining from incremental datasets," *Data Sci. Eng.*, vol. 9, no. 1, pp. 73–87, 2024.
[32] U. Yun, H. Kim, H. Kim, and S. Park, "Efficient fuzzy-based high utility pattern computing and analyzing approach with temporal properties," *Appl. Soft Comput.*, vol. 172, 2025, Art. no. 112902.
[33] M. Han, W. Yang, Z. Dai, S. Yang, and S. Zhu, "A review of meta-heuristic high utility patterns mining methods," *Knowl. Inf. Syst.*, vol. 67, pp. 5469–5510, 2025.
[34] X. Dong et al., "DS_HURNSP: An effective method for mining high utility repeated negative sequential patterns from data streams," *Inf. Process. Manag.*, vol. 63, no. 4, 2026, Art. no. 104637.
[35] N. T. Tung, T. D. D. Nguyen, L. T. T. Nguyen, D.-L. Vu, P. Fournier-Viger, and B. Vo, "Mining cross-level high utility itemsets in unstable and negative profit databases," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 9, pp. 5420–5435, Sep. 2025.
[36] H. Kim et al., "Efficient approach of high average utility pattern mining with indexed list-based structure in dynamic environments," *Inf. Sci.*, vol. 657, 2024, Art. no. 119924.
[37] M. Geng et al., "Mining high average utility nonoverlapping patterns from sequential database," *ACM Trans. Intell. Syst. Technol.*, vol. 17, no. 1, pp. 15:1–15:24, Jan. 2026.
[38] D. Lan et al., "TK-RNSP: Efficient top-k repetitive negative sequential pattern mining," *Inf. Process. Manag.*, vol. 62, no. 3, 2025, Art. no. 104077.
[39] X. Wan and X. Han, "Efficient top-k frequent itemset mining on massive data," *Data Sci. Eng.*, vol. 9, no. 2, pp. 177–203, 2024.
[40] T. You, Y. Sun, Y. Zhang, J. Chen, P. Zhang, and M. Yang, "Accelerated frequent closed sequential pattern mining for uncertain data," *Expert Syst. Appl.*, vol. 204, 2022, Art. no. 117254.
[41] L. Yan, S. Zhang, L. Guo, J. Liu, and X. Wu, "NetNMSP: Nonoverlapping maximal sequential pattern mining," *Appl. Intell.*, vol. 52, pp. 9861–9884, 2022.
[42] S. Liang, L. Chen, W. Gan, P. S. Yu, and S. Zhao, "Targeted mining of time-interval related patterns," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 37, no. 2, pp. 937–950, Feb. 2026.
[43] H. Yan, F. Li, M. C. Hsieh, and M. T. Wu, "High-utility sequential pattern mining in incremental database," *J. Supercomput.*, vol. 81, no. 1, pp. 1–34, 2025.
[44] Y. Wu et al., "OIP-Miner: One-off incremental sequential pattern mining with forgetting factor," *Sci. China Inf. Sci.*, vol. 68, no. 10, 2025, Art. no. 209101.
[45] M. Han, N. Zhang, L. Wang, X. Li, and H. Cheng, "Mining closed high utility patterns with negative utility in dynamic databases," *Appl. Intell.*, vol. 53, no. 10, pp. 11750–11767, 2022.
[46] T. Kieu, B. Vo, T. Le, Z.-H. Deng, and B. Le, "Mining top-k co-occurrence items with sequential pattern," *Expert Syst. Appl.*, vol. 85, pp. 123–133, 2017.
[47] J. Wang, S. Fang, C. Liu, J. Qin, X. Li, and Z. Shi, "Top-k closed co-occurrence patterns mining with differential privacy over multiple streams," *Future Gener. Comput. Syst.*, vol. 111, pp. 339–351, 2020.
[48] X. Liu, X. Niu, and P. Fournier-Viger, "Fast top-k association rule mining using rule generation property pruning," *Appl. Intell.*, vol. 51, no. 4, pp. 2077–2093, 2021.
[49] X. Ao, H. Shi, J. Wang, L. Zuo, H. Li, and Q. He, "Large-scale frequent episode mining from complex event sequences with hierarchies," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 4, 2019, Art. no. 36.
[50] Y. Wu et al., "OPR-Miner: Order-preserving rule mining for time series," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 11, pp. 11722–11735, Nov. 2023.
[51] R. Wang et al., "Review on mining data from multiple data sources," *Pattern Recognit. Lett.*, vol. 109, pp. 120–128, 2018.
[52] S. Saleti and R. B. Subramanyam, "A novel mapreduce algorithm for distributed mining of sequential patterns using co-occurrence information," *Appl. Intell.*, vol. 49, no. 1, pp. 150–171, Jan. 2019.
[53] W. Gan, J. C.-W. Lin, J. Zhang, P. Fournier-Viger, H.-C. Chao, and P. S. Yu, "Fast utility mining on sequence data," *IEEE Trans. Cybern.*, vol. 51, no. 2, pp. 487–500, Feb. 2021.
[54] Y. Wu et al., "HANP-Miner: High average utility nonoverlapping sequential pattern mining," *Knowl. Based Syst.*, vol. 229, 2021, Art. no. 107361.

**Yan Li** received the PhD degree in management science and engineering from Tianjin University, Tianjin, China. She is an associate professor with the Hebei University of Technology. She has authored or coauthored more than 20 research papers in some journals, such as *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Cybernetics*, *ACM Transactions on Knowledge Discovery from Data*, *Information Sciences*, and *Applied Intelligence* Her current research interests include data mining and supply chain management.

**Mengyao He** is currently working toward the master's degree with the Management Science and Engineering, Hebei University of Technology. Her current research interests include data mining and machine learning.

**Jianguo Wei** (Senior Member, IEEE) received the MS degree from Tianjin University, Tianjin, China, in 2002, and the PhD degree from the Japan Advanced Institute of Science and Technology, Nomi, Japan, in 2007. He is currently a professor with the College of Intelligence and Computing, Tianjin University. His research interests include speech signal processing and machine learning.

**Youxi Wu** (Senior Member, IEEE) received the PhD degree in Theory and New Technology of Electrical Engineering from the Hebei University of Technology, Tianjin, China. He is currently a PhD supervisor and a professor with the Hebei University of Technology. He has published more than 30 research papers in some journals, such as *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Cybernetics*, *ACM Transactions on Knowledge Discovery from Data*, *ACM Transactions on Management Information Systems*, *Science China Information Sciences*, *Information Sciences*, *Journal of Computer Science and Technology*, *Knowledge-based Systems*, *Expert Systems With Applications*, *Journal of Information Science*, *Neurocomputing*, and *Applied Intelligence*. He is a distinguished member of CCF. His current research interests include data mining and machine learning.